



RAPPORT DE STAGE

En vue de l'obtention du diplôme de Licence en Business Computing

Parcours Business Intelligence

Réalisé par : **Oussema Korbosli**

**« Conception et développement d'un système intelligent de gestion des rôles
et de pointage des employés avec tableau de bord décisionnel »**

Du 16 février au 16 juin 2026

Maître de stage : Mr. Melek Aziz Elmoula

Encadrant académique : Mr. Heni Abidi

Année universitaire 2025/2026

Résumé

Réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne**, ce projet de fin d'études porte sur la conception et le développement d'un **système intelligent de gestion des rôles et de pointage des employés**, doté d'un tableau de bord décisionnel (Business Intelligence). L'application centralise la gestion des agences, des employés, des clients et des objectifs commerciaux, gère les rôles et la sécurité des accès, assure le **pointage (présence) des employés**, et formalise les opérations sensibles au moyen de circuits de validation. Développée avec la plateforme **Jakarta EE 10** (Servlets, JSP, EJB, JPA/Hibernate) au-dessus d'une base **Oracle**, elle est complétée par un volet décisionnel : une chaîne **ETL** (Python) consolide les données dans un datamart qui alimente des vues exposées à **Microsoft Power BI**, un tableau de bord embarqué (effectifs, présence, production) et une **segmentation des agences** par apprentissage non supervisé (clustering). La solution améliore la centralisation des données, la traçabilité des opérations et l'aide à la décision.

Mots-clés : gestion des rôles, pointage des employés, Business Intelligence, ETL, aide à la décision, apprentissage automatique, clustering, Jakarta EE, Oracle, Power BI.

Abstract

Carried out within **BTK – Banque Tuniso-Koweïtienne**, this end-of-studies project deals with the design and development of an **intelligent system for managing employee roles and time-tracking (attendance)**, featuring a business-intelligence dashboard. The application centralizes the management of branches, employees, customers and commercial objectives, handles roles and access security, records **employee attendance (clock-in/clock-out)**, and formalizes sensitive operations through approval workflows. Built on the **Jakarta EE 10** platform (Servlets, JSP, EJB, JPA/Hibernate) over an **Oracle** database, it is complemented by a decision-support layer : a Python **ETL** pipeline consolidates data into a datamart that feeds analytical views exposed to **Microsoft Power BI**, an embedded dashboard (headcount, attendance, production) and a **branch segmentation** based on unsupervised machine learning (clustering). The solution

improves data centralization, operation traceability and decision-making.

Key words : role management, employee time-tracking, Business Intelligence, ETL, decision support, machine learning, clustering, Jakarta EE, Oracle, Power BI.

Dédicaces

Je dédie ce modeste travail :

*À mes chers **parents**,
pour leur amour, leurs sacrifices et leur soutien sans faille tout au long de mon parcours. Aucun
mot ne saurait exprimer ma gratitude.*

*À ma **famille**,
pour ses encouragements constants et sa présence dans les moments importants.*

*À mes **amis** et à mes **camarades**,
pour les bons moments partagés et leur soutien tout au long de ces années d'études.*

*À tous mes **enseignants**,
pour la qualité de la formation et le savoir qu'ils ont su me transmettre.*

À tous ceux qui m'ont aidé, de près ou de loin, à réaliser ce travail.

Oussema Korbosli

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie chaleureusement mon encadrant académique, **Mr. Heni Abidi**, pour ses conseils avisés, sa disponibilité et son suivi rigoureux tout au long de ce projet.

Mes remerciements s'adressent également à mon maître de stage, **Mr. Melek Aziz Elmoula**, Data Scientist au sein de la **BTK – Banque Tuniso-Koweïtienne**, pour la confiance accordée, l'accompagnement et le partage de l'expérience métier.

Je tiens aussi à remercier l'ensemble du corps enseignant et administratif de l'**Esprit School of Business** pour la qualité de la formation dispensée durant mon cursus en Licence Business Computing, parcours Business Intelligence.

Enfin, j'exprime ma reconnaissance à ma famille et à mes amis pour leur soutien constant et leurs encouragements.

Table des matières

Dédicaces	i
Remerciements	ii
Liste des acronymes	xi
Introduction générale	1
1 Cadre général du projet	2
1.1 Cadre du projet	2
1.2 Présentation de l'organisme d'accueil	3
1.2.1 Missions	4
1.2.2 Objectifs	4
1.3 Étude de l'existant	4
1.3.1 Description de la situation existante	4
1.3.2 Critique de la situation existante	5
1.3.3 Problématique	5
1.3.4 Solution proposée	5
1.3.5 Justification du choix de la solution	5
1.4 Méthodologie et planification du projet	6
1.4.1 Démarche de développement	6
1.4.2 Planification	6

2	Analyse et spécification des besoins	8
2.1	Identification des acteurs	8
2.2	Spécification des besoins	9
2.2.1	Besoins fonctionnels	9
2.2.2	Besoins non fonctionnels	9
2.3	Diagramme de cas d'utilisation global	10
2.4	Description textuelle des cas d'utilisation	10
2.4.1	Authentification et sécurité	10
2.4.2	Gestion du référentiel : agences et employés	13
2.4.3	Gestion commerciale : clients et objectifs	13
2.4.4	Pointage et workflows de demandes	16
2.4.5	Volet décisionnel	16
3	Conception	17
3.1	Architecture du système	17
3.1.1	Architecture globale	17
3.1.2	Architecture physique	17
3.1.3	Architecture logique	19
3.2	Diagramme de classes	20
3.3	Modèle physique de données	22
3.4	Sécurité et contrôle d'accès	22
3.4.1	Authentification et session	22
3.4.2	Rôles et matrice des droits	23
3.5	Diagrammes de séquence	24
3.5.1	S'authentifier	24
3.5.2	Créer une agence	25

3.5.3	Gérer un client	25
3.5.4	Pointer un employé	25
3.5.5	Valider une demande	27
3.6	Cycle de vie d'une demande	27
3.6.1	Diagramme d'états-transitions	27
3.6.2	Diagramme d'activité	28
4	Réalisation	30
4.1	Environnement de travail	30
4.1.1	Environnement matériel	30
4.1.2	Outil de rédaction du rapport	30
4.1.3	Environnement logiciel	31
4.2	Interfaces de l'application	33
4.2.1	Authentification	33
4.2.2	Gestion des agences et des employés	33
4.2.3	Gestion des clients et des objectifs	33
4.2.4	Pointage des employés	36
4.3	Déploiement de l'application	37
4.4	Tests et validation	38
5	Volet décisionnel	39
5.1	Chaîne décisionnelle et processus ETL	39
5.2	Modèle en étoile de l'entrepôt	39
5.3	Mise en œuvre de la chaîne ETL en Python	41
5.3.1	Export des données sources	41
5.3.2	Agrégation et chargement	42
5.4	Tableau de bord et restitution Power BI	43

5.5	Rapport décisionnel sous Power BI	44
5.5.1	Indicateurs et mesures (DAX)	44
5.5.2	Analyse des clients	45
5.5.3	Performance commerciale	45
5.5.4	Suivi de la présence	45
5.5.5	Navigation et interactivité	47
5.6	Segmentation des agences par apprentissage non supervisé	47
5.6.1	Données et variables	47
5.6.2	Prétraitement	48
5.6.3	Détermination du nombre de clusters	48
5.6.4	Comparaison des modèles de clustering	49
5.6.5	Résultats et interprétation	49
5.6.6	Apport décisionnel	50
	Conclusion générale et perspectives	52
	Nétographie	53
A	Annexes	54
A.1	Chaîne ETL — extraction, transformation, chargement (Python)	54
A.2	Segmentation des agences (Python, scikit-learn)	55
A.3	Vue décisionnelle (SQL)	56
A.4	Dépôt du code source	57

Table des figures

1.1	Logo de la BTK – Banque Tuniso-Koweïtienne	3
1.2	Planning prévisionnel du projet (diagramme de Gantt)	6
2.1	Diagramme de cas d'utilisation global	11
3.1	Architecture globale du système	18
3.2	Architecture physique (diagramme de déploiement 3-tiers)	19
3.3	Architecture logique en couches de l'application	20
3.4	Diagramme de classes du modèle métier	21
3.5	Schéma relationnel de la base de données	22
3.6	Diagramme de séquence – Authentification	24
3.7	Diagramme de séquence – Création d'une agence	25
3.8	Diagramme de séquence – Gestion d'un client	26
3.9	Diagramme de séquence – Pointage d'un employé	26
3.10	Diagramme de séquence – Validation d'une demande	27
3.11	Diagramme d'états-transitions – Cycle de vie d'une demande	28
3.12	Diagramme d'activité – Traitement d'une demande	29
4.1	Logo de « TeX »	30
4.2	Logo de « Java (JDK 21) »	31
4.3	Logo de « Jakarta EE »	31
4.4	Logo de « Hibernate »	31
4.5	Logo d'« Oracle Database »	31
4.6	Logo de « WildFly / JBoss »	32

4.7	Logo d'« Apache Maven »	32
4.8	Logo d'« IntelliJ IDEA »	32
4.9	Logos de « Git » et « GitHub »	32
4.10	Logo d'« ApexCharts »	32
4.11	Logo de « Microsoft Power BI »	33
4.12	Logos de « Python », « pandas » et « scikit-learn »	33
4.13	Interface d'authentification	34
4.14	Interface de gestion des agences (sélection et consultation des membres) . . .	34
4.15	Interface de gestion des employés (ajout, filtres et liste)	35
4.16	Interface de gestion des clients (portefeuille BTK)	35
4.17	Suivi des objectifs commerciaux par agence et gestionnaire	36
4.18	Interface du module de pointage des employés	36
5.1	Chaîne décisionnelle : du système opérationnel au datamart et à sa restitution	40
5.2	Modèle décisionnel en étoile (vues V_BI_*)	41
5.3	Tableau de bord décisionnel embarqué de l'application	43
5.4	Rapport Power BI — page « Clients » (segmentation et territoire)	45
5.5	Rapport Power BI — page « Commercial » (objectifs et production)	46
5.6	Rapport Power BI — page « Présence » (assiduité et suivi RH)	46
5.7	Choix du nombre de clusters : méthode du coude et score de silhouette	48
5.8	Comparaison des modèles selon le score de silhouette	49
5.9	Segmentation des agences (K-Means) projetée par PCA	50

Liste des tableaux

1.1	Fiche signalétique de la BTK	3
2.1	Identification des acteurs	8
2.2	Besoins non fonctionnels	9
2.3	Description textuelle du cas d'utilisation « S'authentifier »	10
2.4	Description textuelle du cas d'utilisation « Gérer les rôles et les droits d'accès »	12
2.5	Description textuelle du cas d'utilisation « Gérer les agences »	12
2.6	Description textuelle du cas d'utilisation « Gérer les employés »	12
2.7	Description textuelle du cas d'utilisation « Gérer les clients »	13
2.8	Description textuelle du cas d'utilisation « Suivre les objectifs commerciaux »	13
2.9	Description textuelle du cas d'utilisation « Pointer la présence »	14
2.10	Description textuelle du cas d'utilisation « Valider une demande »	14
2.11	Description textuelle du cas d'utilisation « Alimenter le datamart (ETL) »	14
2.12	Description textuelle du cas d'utilisation « Consulter le tableau de bord »	15
2.13	Description textuelle du cas d'utilisation « Segmenter les agences »	15
3.1	Description des principales tables de la base	23
3.2	Matrice des droits d'accès par rôle	23
4.1	Principaux scénarios de test fonctionnel	38
5.1	Indicateurs de performance (KPI) du volet décisionnel	41
5.2	Principales mesures DAX du rapport Power BI	44
5.3	Variables de segmentation (par agence)	48

5.4	Comparaison des modèles de clustering	49
5.5	Profil moyen des segments d'agences	50
5.6	Recommandations de pilotage par segment	51

Liste des acronymes

Acronyme	Signification
BTk	Banque Tuniso-Koweïtienne
BI	Business Intelligence (informatique décisionnelle)
ETL	Extract, Transform, Load
IA	Intelligence Artificielle
ML	Machine Learning (apprentissage automatique)
ACP	Analyse en Composantes Principales (PCA)
ORM	Object-Relational Mapping
KPI	Key Performance Indicator (indicateur clé de performance)
PFE	Projet de Fin d'Études
UML	Unified Modeling Language
JPA	Jakarta Persistence API
EJB	Enterprise Java Beans
JSP	Jakarta Server Pages
JSTL	Jakarta Standard Tag Library
JTA	Jakarta Transaction API
JDBC	Java Database Connectivity
MVC	Modèle-Vue-Contrôleur
SGBD	Système de Gestion de Base de Données
WAR	Web Application Archive

Acronyme	Signification
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol

Introduction générale

La transformation numérique du secteur bancaire impose aux établissements de disposer de systèmes d'information capables, d'une part, de centraliser la gestion de leur réseau d'agences et, d'autre part, d'exploiter leurs données pour piloter l'activité. La maîtrise de l'information — effectifs, portefeuille clients, objectifs commerciaux — constitue aujourd'hui un levier déterminant de performance et d'aide à la décision.

C'est dans ce contexte que s'inscrit ce projet de fin d'études, réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne** en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'Esprit School of Business. Le projet consiste à concevoir et réaliser une application web de gestion des agences bancaires, adossée à un volet décisionnel d'aide à la décision.

L'application couvre la gestion des agences, des employés, des clients et des objectifs commerciaux, le **pointage (présence) des employés**, ainsi qu'un ensemble de circuits de validation (workflows) encadrant la création et la modification des données sensibles. À ces fonctionnalités transactionnelles s'ajoute un volet décisionnel : une chaîne **ETL** consolide les données dans un datamart qui alimente des rapports Microsoft Power BI, un tableau de bord embarqué et une **segmentation des agences** par apprentissage automatique.

Ce rapport, qui rend compte d'un projet conduit selon une démarche itérative et incrémentale, est organisé en cinq chapitres. Le **premier chapitre** présente le cadre général du projet : l'organisme d'accueil, l'étude de l'existant, la problématique et la démarche adoptée. Le **deuxième chapitre** est consacré à l'analyse et à la spécification des besoins (acteurs, besoins fonctionnels et non fonctionnels, cas d'utilisation). Le **troisième chapitre** détaille la conception : architecture, diagramme de classes et scénarios dynamiques. Le **quatrième chapitre** présente la réalisation : l'environnement de travail et les interfaces de l'application. Le **cinquième chapitre** est dédié au volet décisionnel : la chaîne ETL, le tableau de bord, les rapports Power BI et la segmentation des agences par apprentissage automatique. Une conclusion générale dresse le bilan du travail et ouvre des perspectives d'évolution.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre situe le projet dans son contexte. Il délimite d'abord le cadre du projet, présente ensuite l'organisme d'accueil et ses missions, puis analyse la situation existante afin d'en dégager la problématique et la solution proposée. Il se termine par la présentation de la méthodologie de gestion de projet retenue, le cadre agile **Scrum**.

1.1 Cadre du projet

Le présent travail constitue un **projet de fin d'études** réalisé en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'**Esprit School of Business**. Il s'est déroulé dans le cadre d'un stage au sein de la **BTK – Banque Tuniso-Koweïtienne**, du **16 février au 16 juin 2026**. Le projet porte sur la conception et le développement d'une application web de gestion du réseau d'agences bancaires, adossée à un volet décisionnel d'aide à la décision.

Les objectifs assignés à ce projet sont les suivants :

- **centraliser** la gestion des agences, des employés, des clients et des objectifs commerciaux dans une base unique ;
- **sécuriser** l'accès par une gestion fine des rôles et des droits ;
- **formaliser** les opérations sensibles par des circuits de validation et en assurer la traçabilité ;
- **suivre la présence** des employés au moyen d'un module de pointage ;
- **restituer** des indicateurs d'aide à la décision via une chaîne ETL, un tableau de bord et des rapports Power BI ;

- **valoriser** les données par une segmentation des agences fondée sur l'apprentissage automatique.

1.2 Présentation de l'organisme d'accueil

La **Banque Tuniso-Koweïtienne** (BTK Bank) est une banque universelle de droit tunisien, créée le 25 février 1981 dans le cadre d'une convention de coopération bilatérale entre la Tunisie et le Koweït. Constituée en société anonyme et régie par la loi bancaire n° 2016-48, elle dispose d'un capital social de 200 millions de dinars et a son siège social à Tunis. À travers un réseau d'agences couvrant l'ensemble du territoire tunisien, elle accompagne aussi bien les particuliers que les entreprises.



Figure 1.1 – Logo de la BTK – Banque Tuniso-Koweïtienne

Le tableau 1.1 en résume la fiche signalétique.

Table 1.1 – Fiche signalétique de la BTK

Élément	Renseignement
Dénomination sociale	Banque Tuniso-Koweïtienne (BTK Bank)
Forme juridique	Société Anonyme (S.A.)
Date de création	25 février 1981
Capital social	200 000 000 DT
Siège social	10 bis, Avenue Mohamed V, 1001 Tunis
Secteur d'activité	Banque universelle
Actionnaire de référence	Groupe Elloumi (60 %)
Réseau	5 directions régionales, 6 succursales, 45 agences
Effectif (2024)	≈ 400 collaborateurs

1.2.1 Missions

En tant que banque universelle, la BTK assure un ensemble de missions couvrant les principaux métiers bancaires :

- **Banque de détail** : comptes, cartes, crédits à la consommation et immobiliers, épargne et banque digitale destinés aux particuliers ;
- **Banque des entreprises** : financement de l'investissement et du fonctionnement, financement du commerce extérieur et gestion de trésorerie ;
- **Opérations internationales et de marché** : change, couverture du risque de change et placements ;
- **Innovation et digitalisation** : modernisation des services et offre en ligne.

1.2.2 Objectifs

À travers ces missions, la BTK poursuit plusieurs objectifs stratégiques :

- soutenir le tissu économique en finançant les entreprises et les projets d'investissement ;
- proposer des solutions financières adaptées aux ménages ;
- contribuer à la création d'emplois et à la dynamique de croissance nationale ;
- consolider sa solidité financière et accélérer sa transformation digitale.

1.3 Étude de l'existant

1.3.1 Description de la situation existante

La gestion actuelle du réseau repose en grande partie sur des traitements manuels et des fichiers bureautiques non centralisés. Les informations relatives aux agences, aux employés et aux clients sont saisies et conservées de manière hétérogène ; les demandes de création ou de modification circulent par des canaux informels ; et la présence des employés n'est suivie par aucun dispositif formalisé.

1.3.2 Critique de la situation existante

Cette organisation présente plusieurs limites :

- dispersion et redondance des données, sources d'incohérences ;
- absence de circuit de validation formalisé pour les opérations sensibles ;
- absence de suivi de présence (pointage) des employés ;
- faible traçabilité des actions et des décisions ;
- difficulté à produire des indicateurs consolidés pour le pilotage ;
- gestion des droits d'accès insuffisamment maîtrisée.

1.3.3 Problématique

La principale problématique qui se pose est la suivante : comment **centraliser et fiabiliser** la gestion du réseau d'agences bancaires, **encadrer** les opérations sensibles par des circuits de validation, **assurer le suivi de présence** des employés, et **exploiter** les données ainsi structurées pour fournir aux responsables des indicateurs d'aide à la décision ?

1.3.4 Solution proposée

La solution retenue consiste en une **application web centralisée**, structurée autour d'une base de données Oracle, qui couvre la gestion des agences, des employés, des clients et des objectifs, gère les rôles et la sécurité des accès, assure le **pointage** de présence des employés, et formalise les demandes de création et de modification au moyen de **workflows de validation**. Un **volet décisionnel** complète le dispositif : une chaîne ETL consolide les données dans un datamart qui alimente à la fois Microsoft Power BI, un tableau de bord embarqué et une **segmentation des agences** par apprentissage automatique.

1.3.5 Justification du choix de la solution

Le choix d'une application web centralisée se justifie par plusieurs facteurs : l'**accessibilité** (un simple navigateur suffit, sans installation sur les postes), la **centralisation** des données dans une base unique garantissant leur cohérence, la **sécurité** offerte par une gestion fine des rôles et

des workflows de validation, et l'**ouverture décisionnelle** permise par l'exposition des données à des outils d'analyse et de *reporting*.

1.4 Méthodologie et planification du projet

1.4.1 Démarche de développement

Le périmètre du projet ayant été précisé progressivement au fil des échanges avec l'organisme d'accueil, une **démarche itérative et incrémentale**, d'inspiration *agile*, a été adoptée. Plutôt qu'un enchaînement séquentiel figé, cette approche privilégie des livraisons fonctionnelles successives et une validation régulière avec l'encadrement, ce qui permet d'**adapter la solution** aux retours et de **détecter les risques au plus tôt**. Le travail a été organisé en phases classiques du génie logiciel — analyse et spécification des besoins, conception, réalisation, puis volet décisionnel — reprises comme fil conducteur des chapitres de ce rapport.

1.4.2 Planification

Le projet s'est déroulé sur quatre mois, du 16 février au 16 juin 2026. La figure 1.2 en présente le planning prévisionnel sous forme de diagramme de Gantt, organisé selon les grandes phases du projet.

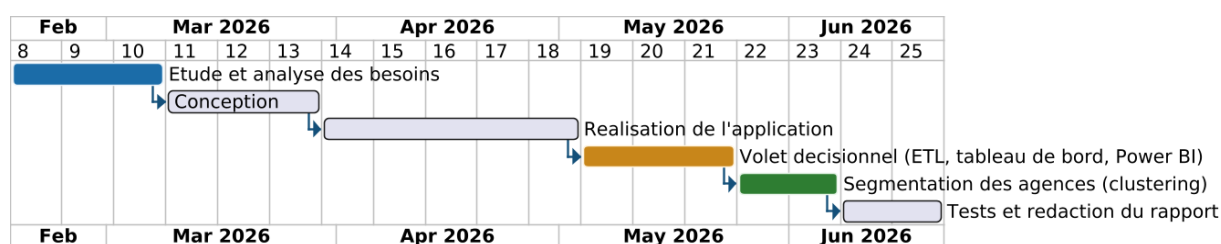


Figure 1.2 – Planning prévisionnel du projet (diagramme de Gantt)

Conclusion

Ce chapitre a posé le cadre du projet : l'organisme d'accueil et ses missions, l'étude de l'existant qui a mis en évidence la problématique, la solution proposée, ainsi que la démarche

et la planification retenues. Le chapitre suivant est consacré à l'**analyse et à la spécification des besoins**.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre est consacré à l'**analyse** du système à réaliser. Il identifie d'abord les acteurs qui interagissent avec l'application, spécifie ensuite les besoins fonctionnels et non fonctionnels, présente le diagramme de cas d'utilisation global, puis détaille par une description textuelle les principaux cas d'utilisation, regroupés par module fonctionnel.

2.1 Identification des acteurs

Trois profils interagissent avec l'application, conformément aux rôles gérés par le module de sécurité (table SECURITY_USERS). Le tableau 2.1 en précise les responsabilités.

Table 2.1 – Identification des acteurs

Acteur	Responsabilités
Administrateur (ADMIN)	Gère les agences, les employés et les clients ; valide les demandes de création de clients et d'employés ; consulte le journal d'audit ; supervise le volet décisionnel.
Directeur commercial (DIRECTEUR_COMMERCIAL)	Soumet les demandes de création (clients, employés) ; valide les demandes de modification des données des employés ; consulte les objectifs et les indicateurs.
Utilisateur (USER)	Consulte les données autorisées, effectue son pointage et soumet des demandes de modification de ses propres informations.

2.2 Spécification des besoins

2.2.1 Besoins fonctionnels

Le système doit permettre :

- l'authentification sécurisée et la gestion des rôles et des droits d'accès (ADMIN, DIRECTEUR_COMMERCIAL, USER);
- la gestion (CRUD) des agences, des employés et des clients;
- la consultation des objectifs commerciaux et le suivi de la production;
- le **pointage (présence) des employés** : saisie de l'heure d'arrivée et de départ, statut (présent / retard / absent), en mode automatique (bouton) ou manuel (administrateur);
- la soumission et la validation des demandes (clients, employés, modifications) selon le cycle EN_ATTENTE → EN_COURS → EFFECTUE/REFUSE;
- la notification des demandes en attente et la journalisation des actions;
- la consolidation des données par une chaîne **ETL** et la restitution d'indicateurs via un tableau de bord et Power BI;
- la **segmentation des agences** par apprentissage non supervisé.

2.2.2 Besoins non fonctionnels

Le tableau 2.2 récapitule les besoins non fonctionnels.

Table 2.2 – Besoins non fonctionnels

Besoin	Description
Sécurité	Authentification, gestion des rôles et protection des opérations sensibles par des workflows de validation.
Fiabilité	Intégrité des données garantie par le SGBD et les transactions JTA.
Performance	Temps de réponse acceptable sur les volumes manipulés.
Ergonomie	Interface claire et cohérente, navigation par menu latéral.
Maintenabilité	Architecture en couches, code modulaire et faiblement couplé.
Portabilité	Application Jakarta EE standard, déployable sur serveur compatible.

2.3 Diagramme de cas d'utilisation global

La figure 2.1 présente le **diagramme de cas d'utilisation global**, qui représente d'un point de vue fonctionnel les interactions entre les acteurs et les services offerts par le système.

On y distingue les trois acteurs et leurs cas d'utilisation respectifs. Le cas *S'authentifier* est commun à tous; il est relié aux autres par une relation «include», l'accès aux fonctionnalités exigeant une authentification préalable. L'**administrateur** concentre les cas de gestion et de validation; le **directeur commercial** soumet les demandes et consulte les indicateurs; l'**utilisateur** se limite à la consultation, au pointage et aux demandes le concernant.

2.4 Description textuelle des cas d'utilisation

Cette section détaille, sous forme de fiches, les principaux cas d'utilisation, regroupés par module fonctionnel.

2.4.1 Authentification et sécurité

Table 2.3 – Description textuelle du cas d'utilisation « S'authentifier »

Titre	S'authentifier
Objectif	Permettre à l'utilisateur de se connecter à l'application avec son identifiant et son mot de passe.
Acteur	Administrateur / Directeur commercial / Utilisateur
Pré-condition	L'utilisateur doit disposer d'un compte actif dans la table SECURITY_USERS.
Post-condition	Une session est ouverte et l'utilisateur est redirigé vers le tableau de bord correspondant à son rôle.
Scénario nominal	(1) L'utilisateur saisit son identifiant et son mot de passe; (2) le SecurityService vérifie les informations sur la table de sécurité; (3) en cas de succès, une session est ouverte et l'utilisateur est redirigé; (4) sinon, un message d'erreur est affiché.

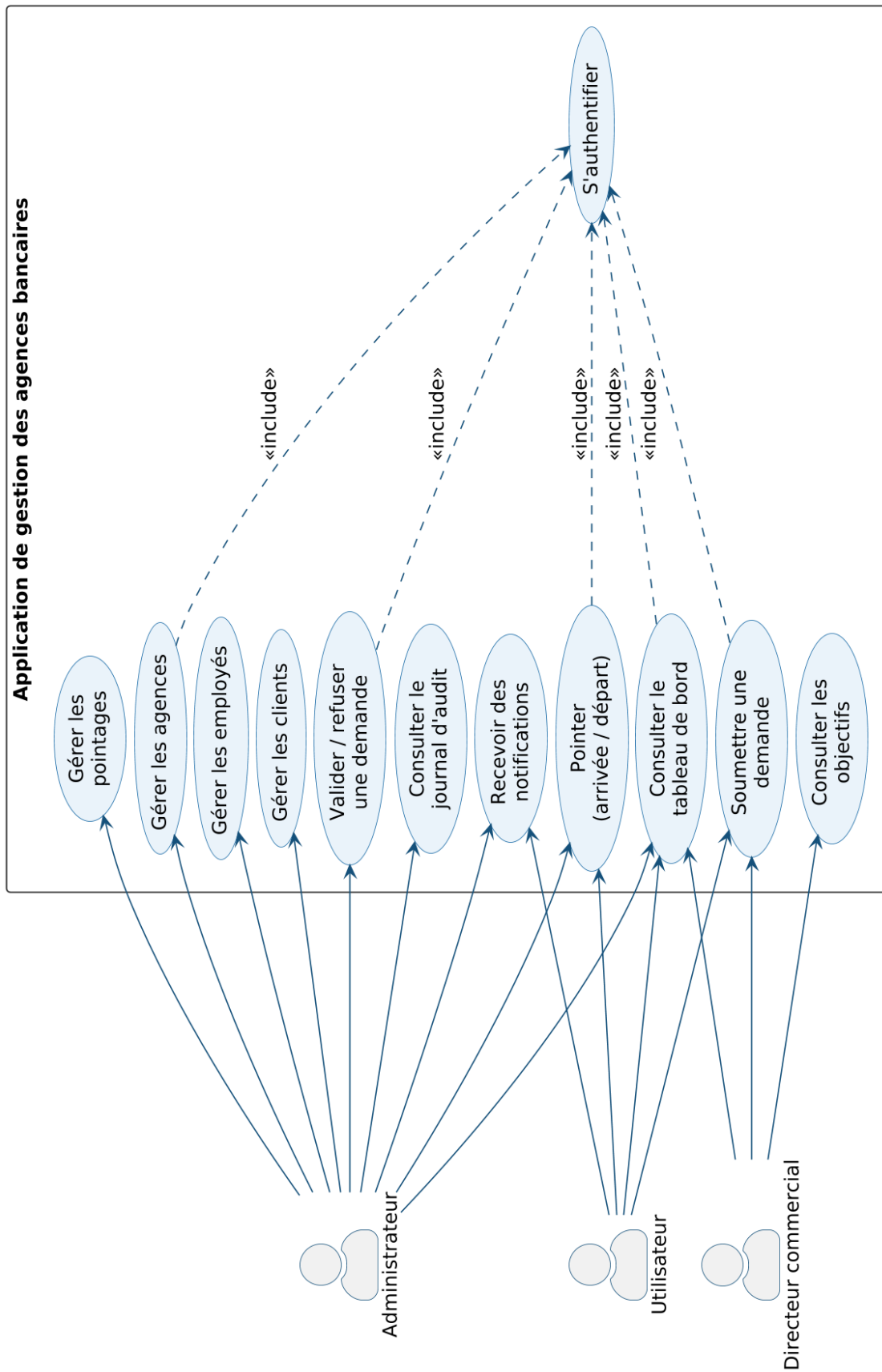


Figure 2.1 – Diagramme de cas d'utilisation global

Table 2.4 – Description textuelle du cas d'utilisation « Gérer les rôles et les droits d'accès »

Titre	Gérer les rôles et les droits d'accès
Objectif	Permettre à l'administrateur d'attribuer un rôle à chaque utilisateur afin de contrôler l'accès aux fonctionnalités.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié avec le rôle ADMIN.
Post-condition	Le rôle est enregistré et les droits d'accès de l'utilisateur sont mis à jour.
Scénario nominal	(1) L'administrateur consulte la liste des utilisateurs ; (2) il sélectionne un utilisateur ; (3) il lui attribue un rôle (ADMIN, DIRECTEUR_COMMERCIAL ou USER) ; (4) le système applique le contrôle d'accès correspondant.

Table 2.5 – Description textuelle du cas d'utilisation « Gérer les agences »

Titre	Gérer les agences
Objectif	Permettre à l'administrateur de créer, consulter, modifier et supprimer les agences du réseau.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié avec le rôle ADMIN.
Post-condition	L'agence créée ou modifiée est enregistrée dans la base et la liste est mise à jour.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des agences ; (2) il sélectionne une action (créer, modifier, supprimer) ; (3) il saisit ou met à jour les informations de l'agence ; (4) l'AgenceService persiste la modification et la liste est rafraîchie.

Table 2.6 – Description textuelle du cas d'utilisation « Gérer les employés »

Titre	Gérer les employés
Objectif	Permettre à l'administrateur de gérer les employés et de les rattacher à leur agence.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié et l'agence de rattachement doit exister.
Post-condition	L'employé est enregistré et rattaché à son agence.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des employés ; (2) il saisit les informations de l'employé ; (3) il sélectionne l'agence de rattachement ; (4) l'UtilisateurService enregistre l'employé associé à l'agence.

2.4.2 Gestion du référentiel : agences et employés

2.4.3 Gestion commerciale : clients et objectifs

Table 2.7 – Description textuelle du cas d'utilisation « Gérer les clients »

Titre	Gérer les clients
Objectif	Permettre à l'administrateur de gérer le portefeuille clients et d'affecter chaque client à un gestionnaire.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié et le gestionnaire de rattachement doit exister.
Post-condition	Le client est enregistré et associé à son gestionnaire dans la base.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des clients ; (2) il saisit ou met à jour les informations du client ; (3) il sélectionne le gestionnaire responsable ; (4) le <code>ClientService</code> enregistre le client et met à jour le portefeuille.

Table 2.8 – Description textuelle du cas d'utilisation « Suivre les objectifs commerciaux »

Titre	Suivre les objectifs commerciaux
Objectif	Permettre au directeur commercial de consulter les objectifs et la production par agence.
Acteur	Directeur commercial / Administrateur
Pré-condition	L'acteur doit être authentifié et des objectifs doivent être renseignés pour la période.
Post-condition	Les indicateurs de production (comptes, crédits, épargne) sont restitués par agence.
Scénario nominal	(1) L'acteur accède à l'interface des objectifs ; (2) il sélectionne une agence et une période ; (3) l' <code>ObjectifService</code> agrège la production ; (4) les indicateurs sont affichés.

Table 2.9 – Description textuelle du cas d'utilisation « Pointer la présence »

Titre	Pointer la présence
Objectif	Permettre à l'employé d'enregistrer son arrivée et son départ, avec détermination automatique du statut.
Acteur	Utilisateur (employé)
Pré-condition	L'employé doit être authentifié.
Post-condition	Le pointage du jour est enregistré avec son statut (PRESENT ou RETARD).
Scénario nominal	(1) L'employé clique sur « Pointer » ; (2) le <code>PointageService</code> relève l'heure courante et recherche le pointage du jour ; (3) s'il n'existe pas, il détermine le statut (RETARD au-delà de 9 h) et l'enregistre ; (4) sinon, il met à jour l'enregistrement existant.

Table 2.10 – Description textuelle du cas d'utilisation « Valider une demande »

Titre	Valider une demande
Objectif	Permettre à l'administrateur d'approuver ou de refuser une demande soumise.
Acteur	Administrateur
Pré-condition	Une demande au statut EN_ATTENTE doit exister.
Post-condition	Le statut de la demande évolue vers EFFECTUE ou REFUSE et l'action est journalisée.
Scénario nominal	(1) L'administrateur consulte les demandes en attente ; (2) il sélectionne une demande ; (3) il l'approuve ou la refuse avec un commentaire ; (4) le service met à jour le statut, applique la modification le cas échéant et journalise l'action.

Table 2.11 – Description textuelle du cas d'utilisation « Alimenter le datamart (ETL) »

Titre	Alimenter le datamart (ETL)
Objectif	Consolider les données opérationnelles en un datamart agrégé par agence.
Acteur	Processus ETL / Administrateur
Pré-condition	Les données sources (agences, employés, clients, objectifs, pointage) doivent être disponibles.
Post-condition	Le datamart en étoile est produit et mis à la disposition de la restitution et de la segmentation.
Scénario nominal	(1) L'étape <i>Extract</i> lit les données sources ; (2) l'étape <i>Transform</i> les nettoie et les agrège par agence ; (3) l'étape <i>Load</i> écrit le datamart (<code>dim_agence</code> , <code>fait_agence</code>).

Table 2.12 – Description textuelle du cas d'utilisation « Consulter le tableau de bord »

Titre	Consulter le tableau de bord
Objectif	Restituer aux responsables une vision synthétique de l'activité.
Acteur	Administrateur / Directeur commercial
Pré-condition	L'acteur doit être authentifié.
Post-condition	Les indicateurs clés (effectifs, présence, production) sont affichés.
Scénario nominal	(1) L'acteur se connecte ; (2) le <code>HomeServlet</code> calcule les statistiques ; (3) la page <code>home.jsp</code> affiche les cartes d'indicateurs et les graphiques <code>ApexCharts</code> .

Table 2.13 – Description textuelle du cas d'utilisation « Segmenter les agences »

Titre	Segmenter les agences
Objectif	Regrouper automatiquement les agences en profils homogènes à partir de leurs indicateurs d'activité.
Acteur	Analyste décisionnel / Administrateur
Pré-condition	Le datamart agrégé par agence (<code>fait_agence</code>) doit être disponible.
Post-condition	Les agences sont réparties en segments cohérents, caractérisés et exploitables pour la décision.
Scénario nominal	(1) L'analyste lance la segmentation ; (2) le script standardise les variables ; (3) il détermine le nombre de clusters puis compare les modèles ; (4) il retient le meilleur modèle, projette en PCA et restitue le profil de chaque segment.

2.4.4 Pointage et workflows de demandes

2.4.5 Volet décisionnel

Conclusion

Ce chapitre a identifié les acteurs, spécifié les besoins fonctionnels et non fonctionnels et formalisé les cas d'utilisation du système. Le chapitre suivant traduit cette analyse en une **conception** détaillée : architecture, modèle de classes et scénarios dynamiques.

Chapitre 3

Conception

Introduction

Ce chapitre traduit l'analyse en une **conception** détaillée du système. Il présente d'abord l'architecture (globale, physique et logique), puis le modèle statique du domaine (diagramme de classes) et enfin les scénarios dynamiques des fonctionnalités principales (diagrammes de séquence, d'états-transitions et d'activité).

3.1 Architecture du système

L'architecture définit l'organisation des composants du système et la manière dont ils communiquent. Elle est présentée à trois niveaux : globale, physique et logique.

3.1.1 Architecture globale

L'architecture globale (figure 3.1) offre une vue d'ensemble des composants. Un poste client (navigateur) dialogue en HTTP/HTTPS avec l'application web déployée sur **WildFly**, structurée en couches (présentation, contrôle, métier, persistance) ; celle-ci accède à la base **Oracle** en JDBC. La base alimente, d'une part, **Power BI** via les vues V_BI_* et, d'autre part, la **chaîne décisionnelle Python** (ETL puis segmentation).

3.1.2 Architecture physique

L'architecture physique (figure 3.2) représente le **déploiement** des composants sur les nœuds matériels selon une organisation **3-tiers** : le poste client, le serveur d'applications WildFly

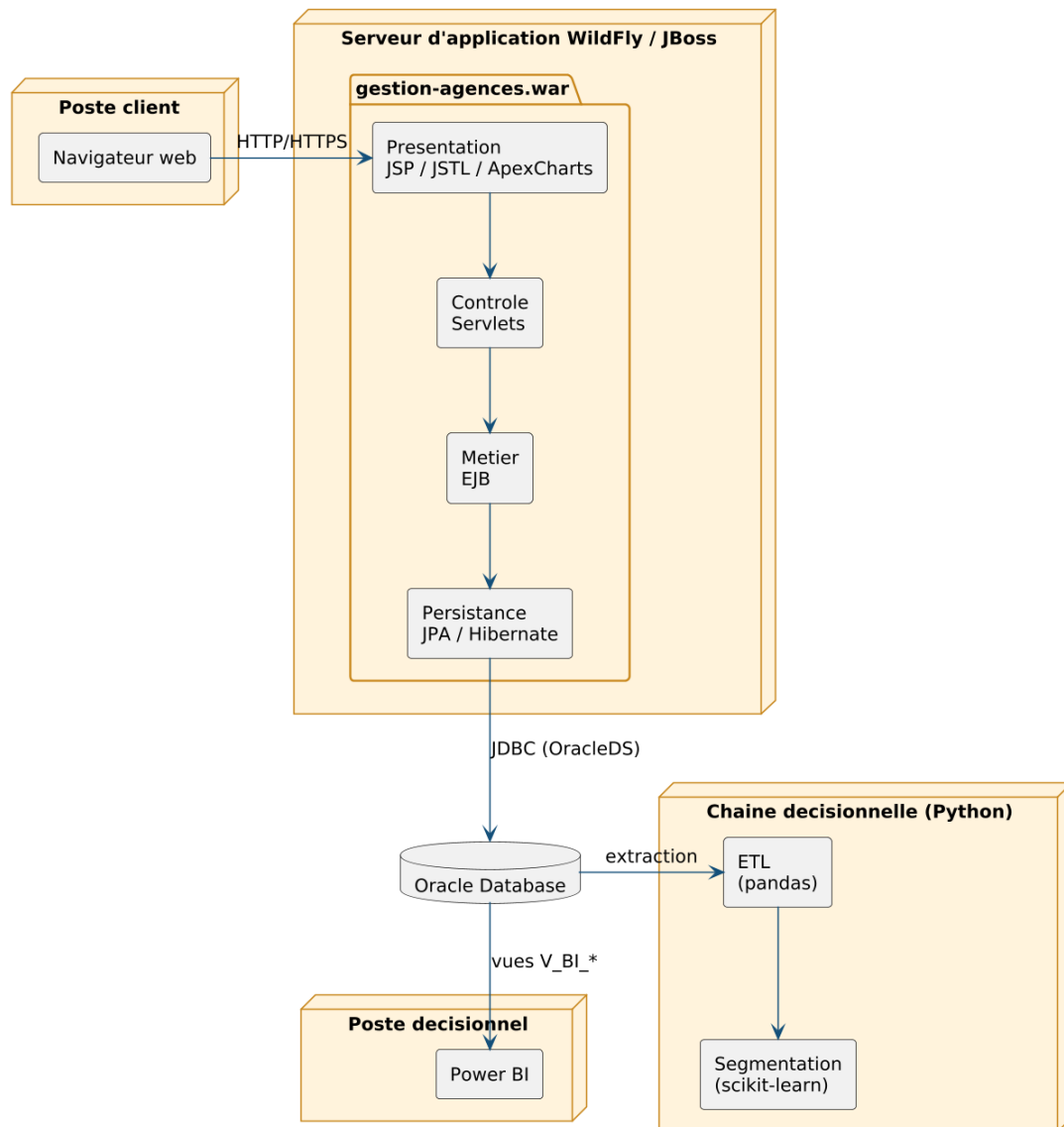


Figure 3.1 – Architecture globale du système

hébergeant `gestion-agences.war` (accès à la base via la source de données JTA OracleDS), le serveur de base de données Oracle (FREEPDB1) et le poste décisionnel exécutant Power BI.

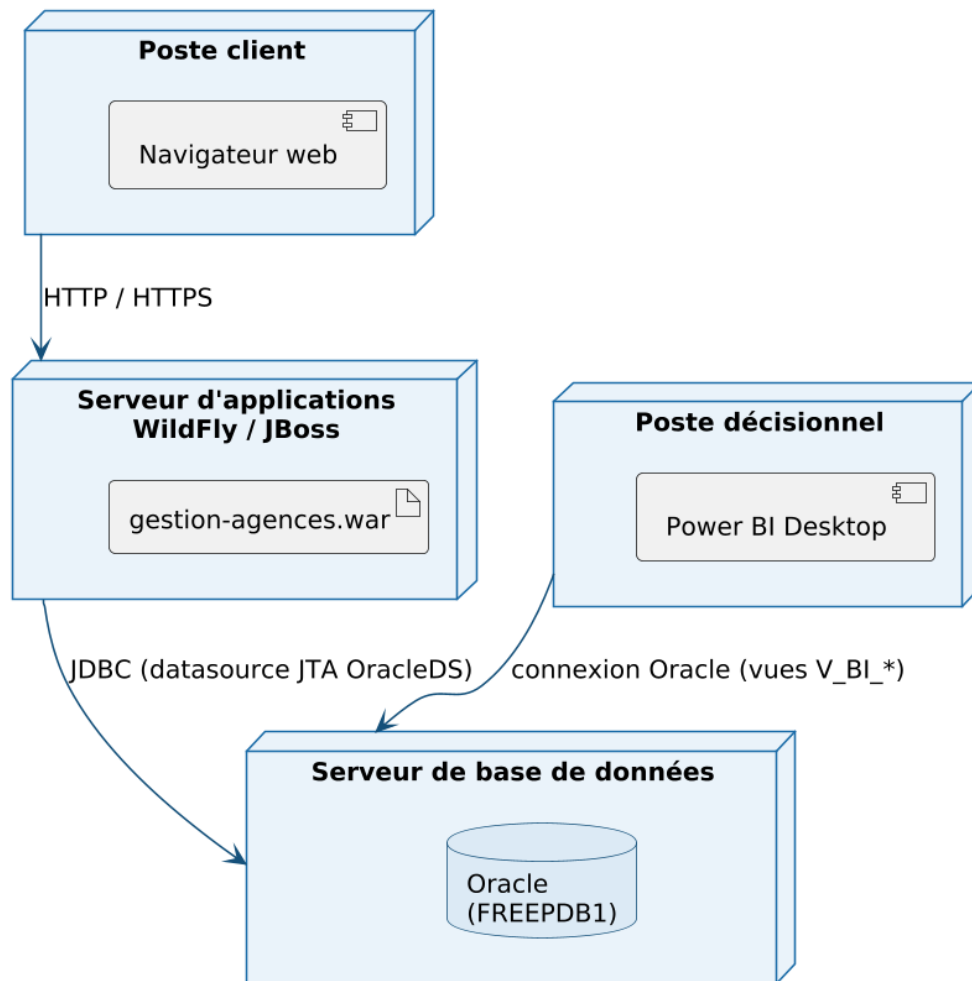
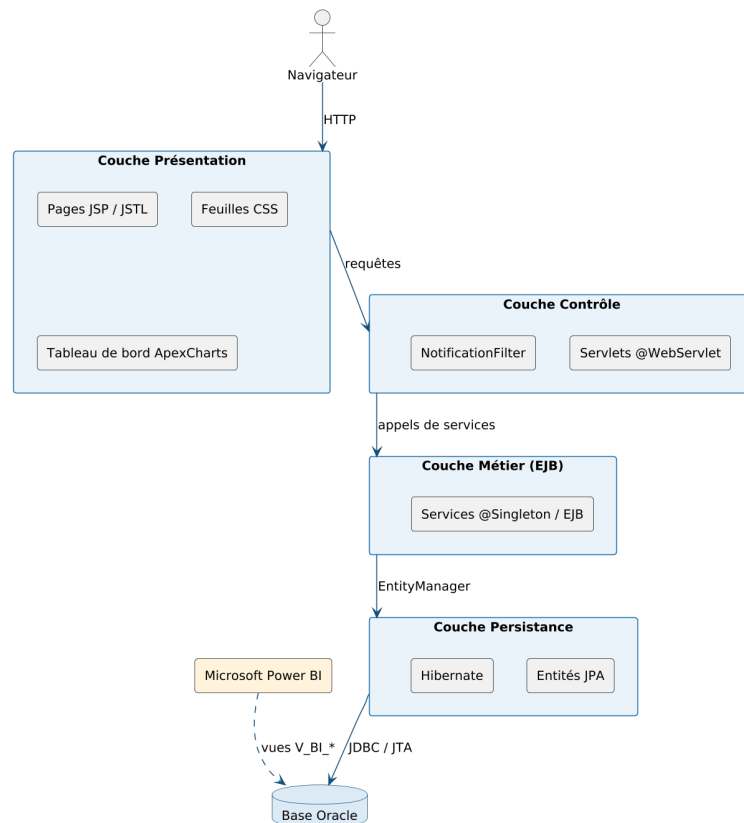


Figure 3.2 – Architecture physique (diagramme de déploiement 3-tiers)

3.1.3 Architecture logique

L'architecture logique (figure 3.3) décompose le système en **couches** faiblement couplées suivant le patron **MVC** : la couche *présentation* (JSP/JSTL, ApexCharts), la couche *contrôle* (Servlets et le filtre `NotificationFilter`), la couche *métier* (services EJB) et la couche *persistance* (entités JPA et Hibernate dialoguant avec Oracle via l'`EntityManager`).



3.2 Diagramme de classes

La figure 3.4 présente le **diagramme de classes** du modèle métier, qui représente les entités persistantes du domaine, leurs attributs et les associations qui les relient.

L'entité Utilisateur occupe une position centrale. Le rattachement d'un Utilisateur, d'un Client ou d'un Objectif à son Agence, ainsi que d'un Client à son Gestionnaire, est matérialisé dans le code par des **clés étrangères** (SK_AGENCE, SK_GESTIONNAIRE). Les entités de demande (DemandeClient, DemandeEmploye, DemandeModification, DemandeAgence), ainsi que JournalAction, Notification et Pointage, référencent un Utilisateur par une association JPA @ManyToOne; DemandeAgence possède en outre une association @ManyToMany avec Utilisateur (utilisateursDemandes). Enfin, SecurityUser est relié à Utilisateur par une dépendance d'authentification. Le modèle ne comporte ni héritage ni composition : les liens sont des associations, dont certaines sont implémentées par des clés étrangères et d'autres par des relations JPA explicites.

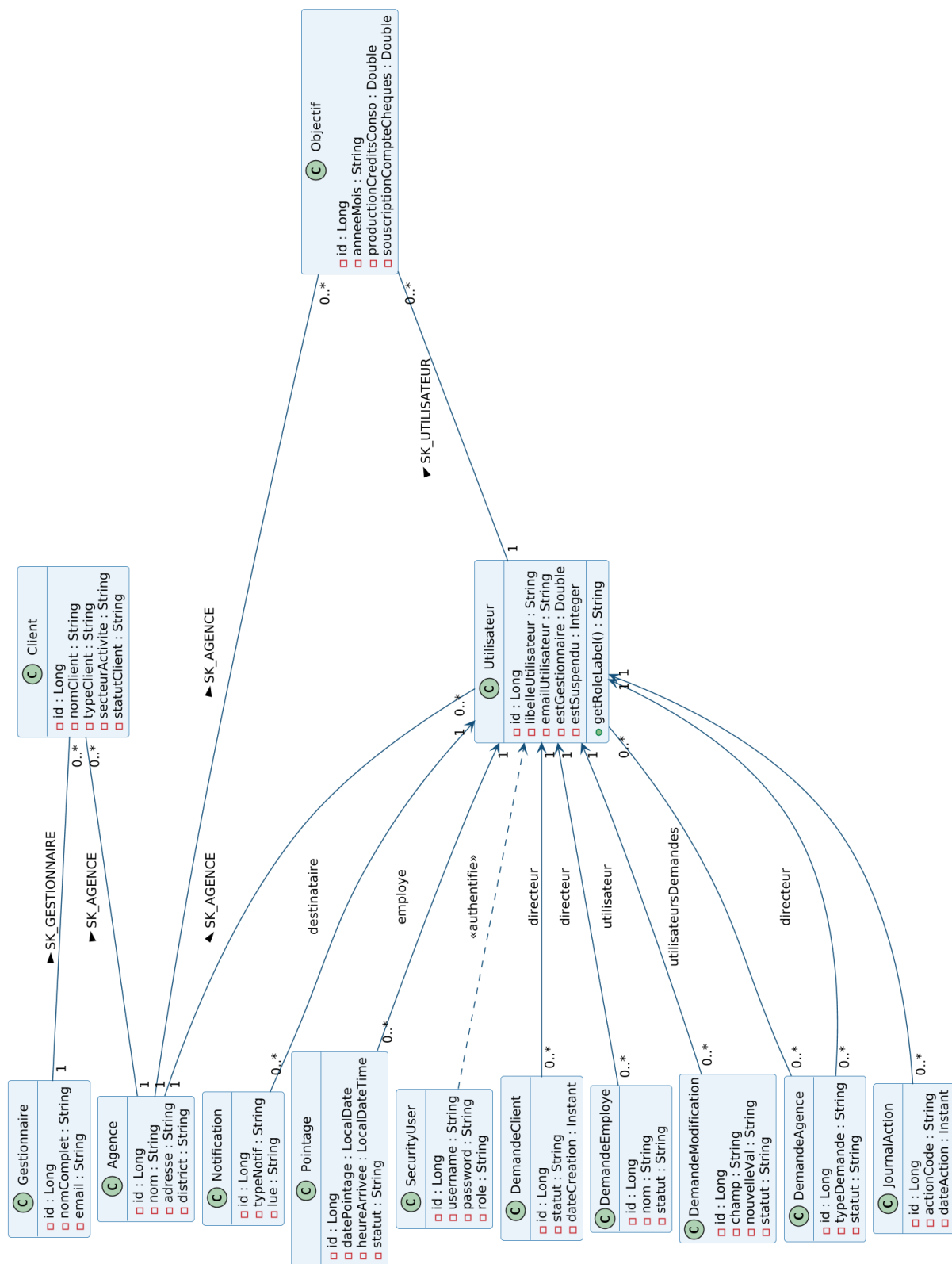


Figure 3.4 – Diagramme de classes du modèle métier

3.3 Modèle physique de données

Au modèle de classes correspond, au niveau de la base de données, un **schéma relationnel** : chaque entité persistante est traduite en une table Oracle, chaque attribut en une colonne et chaque association en une **clé étrangère**. La figure 3.5 présente ce schéma relationnel, obtenu par *mapping* objet-relationnel (JPA/Hibernate) à partir des entités du domaine.

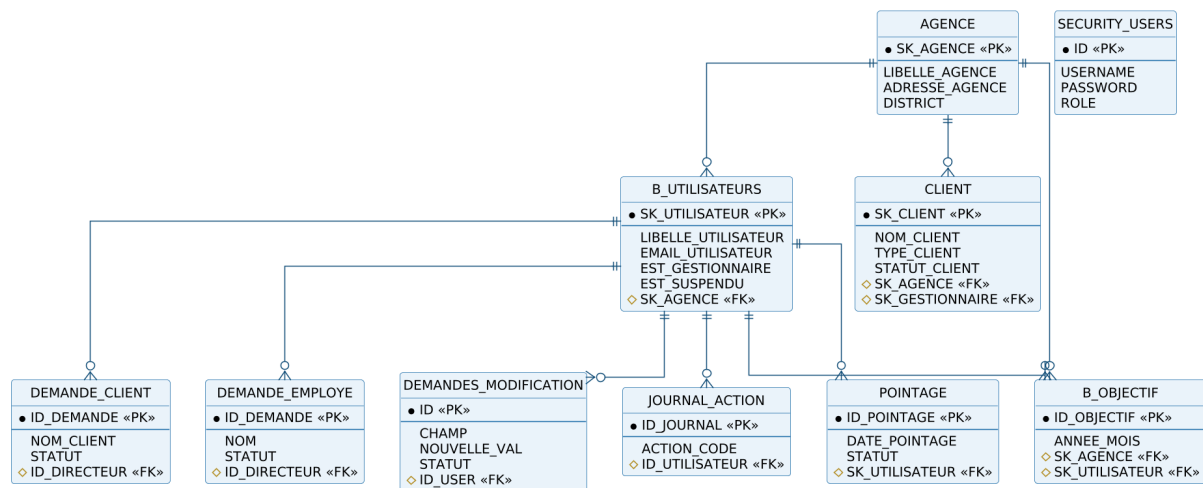


Figure 3.5 – Schéma relationnel de la base de données

La table AGENCE constitue le référentiel central autour duquel gravitent les employés (B_UTILISATEURS), les clients (B_CLIENTS) et les objectifs (B_OBJECTIF), reliés par la clé SK_AGENCE. La table B_UTILISATEURS est à son tour référencée par les pointages, les demandes et le journal d'audit. Le tableau 3.1 décrit le rôle des principales tables du schéma.

3.4 Sécurité et contrôle d'accès

La sécurité de l'application repose sur trois mécanismes complémentaires : l'authentification, la gestion des rôles et le filtrage des requêtes.

3.4.1 Authentification et session

Les comptes sont stockés dans la table SECURITY_USERS (identifiant, mot de passe et rôle). Le LoginServlet délègue la vérification au SecurityService ; en cas de succès, une HttpSession est ouverte et l'Utilisateur métier correspondant y est associé. Le filtre

Table 3.1 – Description des principales tables de la base

Table	Rôle	Clé primaire
AGENCE	Référentiel des agences du réseau (libellé, adresse, district).	SK_AGENCE
B_UTILISATEURS	Employés et utilisateurs de l'application.	SK_UTILISATEUR
B_CLIENTS	Portefeuille clients (type, statut, secteur, adresse).	SK_CLIENT
B_OBJECTIF	Objectifs commerciaux et production par période.	ID_OBJECTIF
POINTAGE	Pointages de présence des employés.	ID_POINTAGE
GESTIONNAIRE	Gestionnaires et rattachement de portefeuille.	SK_UTILISATEUR
SECURITY_USERS	Comptes, mots de passe et rôles (sécurité).	ID
DEMANDE_CLIENT	Demandes de création de clients.	ID_DEMANDE
DEMANDE_EMPLOYE	Demandes de création d'employés.	ID_DEMANDE
DEMANDES_MODIF.	Demandes de modification de données.	ID
JOURNAL_ACTION	Journal d'audit des actions.	ID_JOURNAL
NOTIFICATION	Notifications adressées aux utilisateurs.	ID

NotificationFilter intercepte ensuite les requêtes pour vérifier la présence d'une session valide avant tout accès aux pages protégées, et redirige vers la page de connexion dans le cas contraire.

3.4.2 Rôles et matrice des droits

Trois rôles structurent les droits d'accès. Le tableau 3.2 présente la matrice des permissions par fonctionnalité.

Table 3.2 – Matrice des droits d'accès par rôle

Fonctionnalité	ADMIN	DIR. COMM.	USER
Gestion des agences, employés, clients	Oui	Non	Non
Validation des demandes	Oui	Modif. employés	Non
Soumission de demandes	Oui	Oui	Ses informations
Consultation des objectifs et indicateurs	Oui	Oui	Autorisé
Pointage de présence	Oui	Oui	Oui
Supervision du volet décisionnel	Oui	Non	Non
Consultation du journal d'audit	Oui	Non	Non

Ce cloisonnement garantit qu'un utilisateur ne peut accéder qu'aux fonctionnalités autorisées par son rôle : une tentative d'accès direct à une ressource réservée est bloquée par le contrôle de session et de rôle.

3.5 Diagrammes de séquence

Les diagrammes de séquence décrivent le déroulement dynamique des principales fonctionnalités, en montrant les échanges entre la couche de présentation, les servlets, les services métier et la base de données.

3.5.1 S'authentifier

La figure 3.6 décrit le déroulement dynamique de l'authentification.

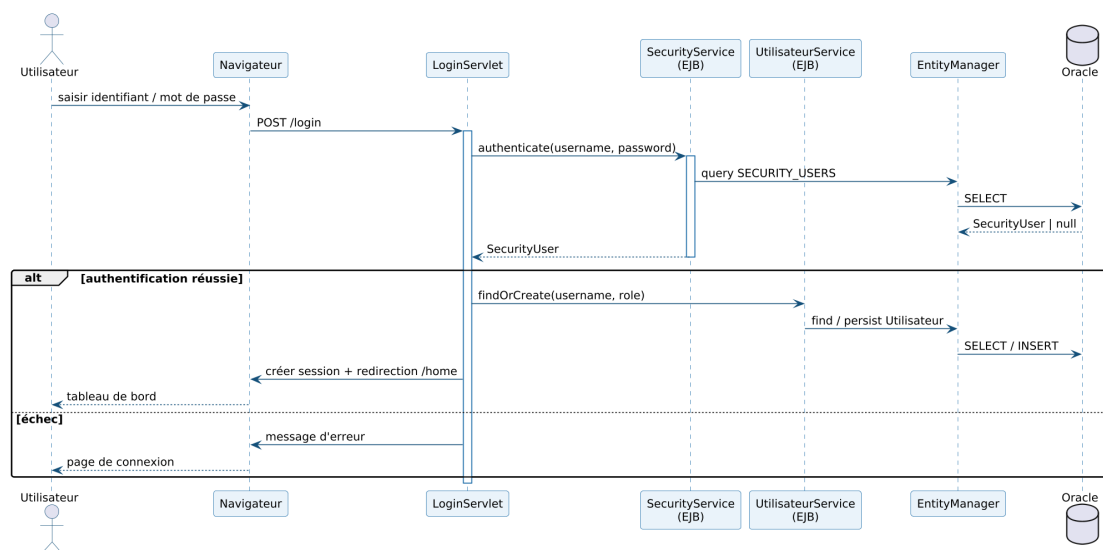


Figure 3.6 – Diagramme de séquence – Authentification

Le LoginServlet reçoit la requête POST /login et délègue la vérification au SecurityService, qui interroge la table SECURITY_USERS via l'EntityManager. Le fragment combiné alt distingue les deux issues : en cas de succès, l'Utilisateur métier est associé (ou créé), une session est ouverte et l'utilisateur est redirigé vers le tableau de bord ; en cas d'échec, un message d'erreur est renvoyé.

3.5.2 Créer une agence

La figure 3.7 décrit le scénario de création d'une agence.

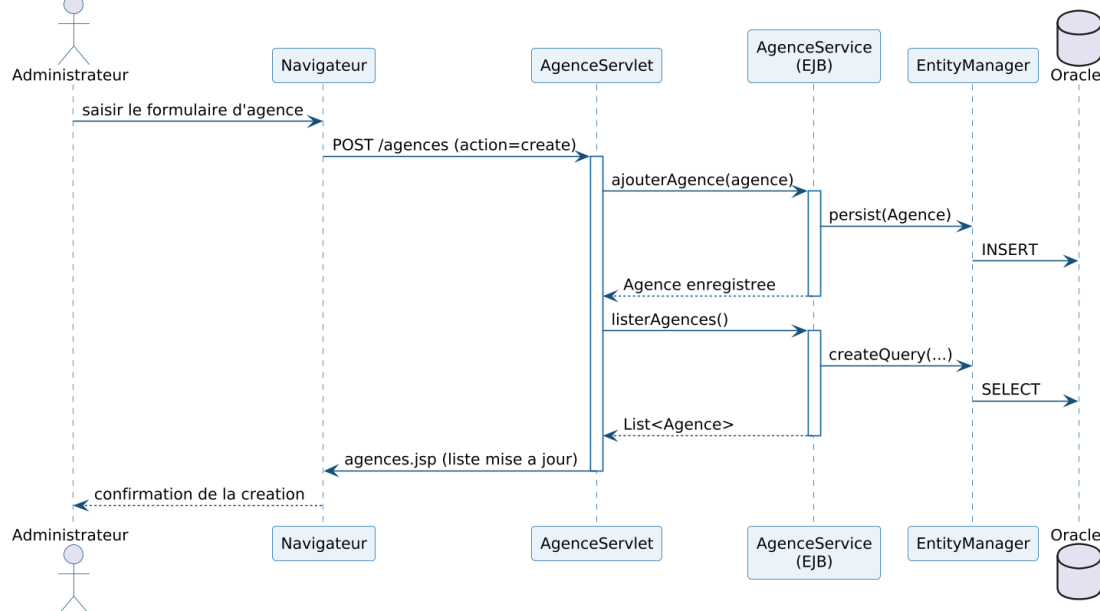


Figure 3.7 – Diagramme de séquence – Création d'une agence

L'AgenceServlet reçoit la requête de création et délègue à l'AgenceService, qui persiste la nouvelle Agence via l'EntityManager (INSERT), puis recharge la liste des agences renvoyée à la page agences.jsp. Le même schéma s'applique aux opérations de modification et de suppression.

3.5.3 Gérer un client

La figure 3.8 décrit l'enregistrement et l'affectation d'un client.

Le ClientServlet transmet la demande au ClientService, qui recherche le Gestionnaire concerné, associe le Client à ce dernier, puis le persiste via l'EntityManager. Le portefeuille mis à jour est renvoyé à la page client.jsp.

3.5.4 Pointer un employé

La figure 3.9 décrit le scénario de pointage.

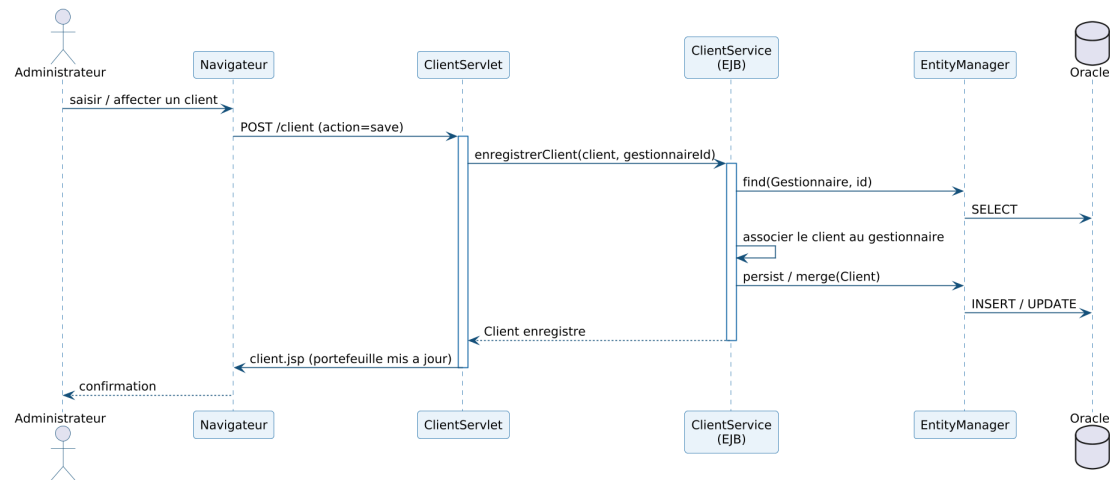


Figure 3.8 – Diagramme de séquence – Gestion d'un client

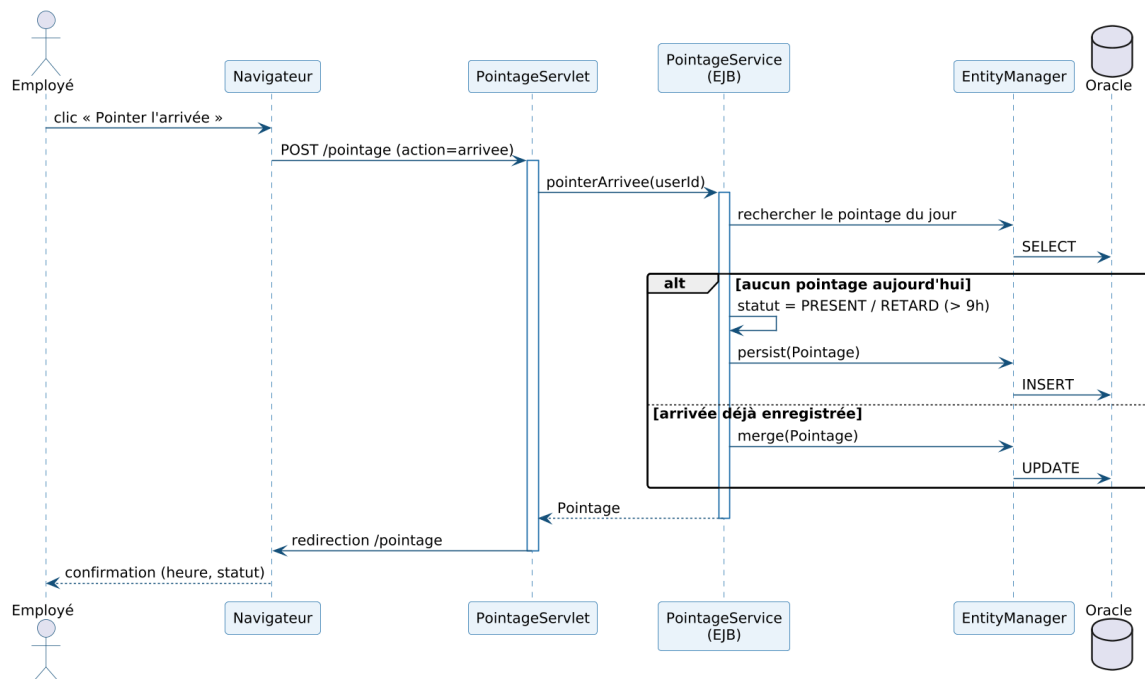


Figure 3.9 – Diagramme de séquence – Pointage d'un employé

Le `PointageServlet` transmet l'action au `PointageService`, qui recherche le pointage du jour. Le fragment `alt` distingue l'INSERT (premier pointage, statut PRESENT/RETARD au-delà de 9 h) de l'UPDATE (pointage déjà existant).

3.5.5 Valider une demande

La figure 3.10 détaille le traitement d'une demande par l'administrateur.

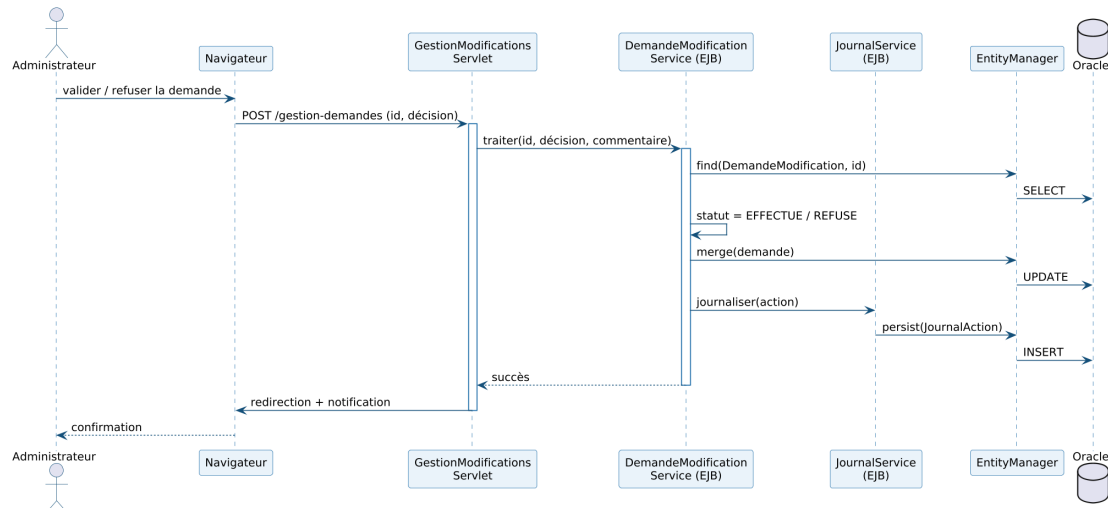


Figure 3.10 – Diagramme de séquence – Validation d'une demande

Le `GestionModificationsServlet` transmet la décision au service métier, qui charge la demande, met à jour son statut, persiste la modification puis journalise l'action via le `JournalService`.

3.6 Cycle de vie d'une demande

Les circuits de validation constituent un mécanisme transverse important. Leur comportement est modélisé par un diagramme d'états-transitions et un diagramme d'activité.

3.6.1 Diagramme d'états-transitions

La figure 3.11 modélise le cycle de vie d'une demande.

Une demande naît à l'état `EN_ATTENTE`, passe en `EN_COURS` lors de sa prise en charge, puis atteint un état final `EFFECTUE` ou `REFUSE`, en cohérence avec la contrainte de la table des

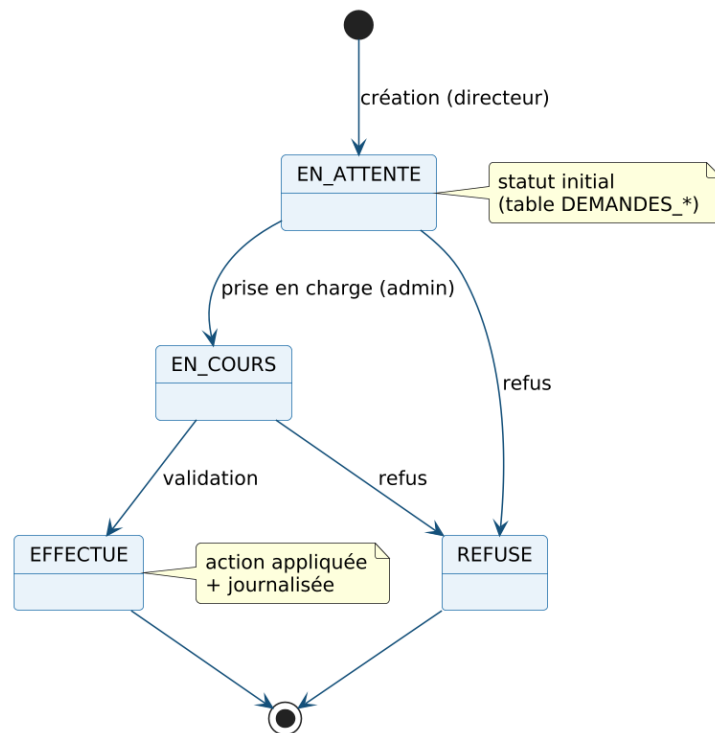


Figure 3.11 – Diagramme d'états-transitions – Cycle de vie d'une demande

demandes.

3.6.2 Diagramme d'activité

La figure 3.12 représente le processus métier de traitement d'une demande.

Le directeur soumet une demande, notifiée à l'administrateur. Le nœud de décision oriente le flux : en cas de validation, le statut passe à EFFECTUE et la modification est appliquée ; en cas de refus, le statut passe à REFUSE avec un commentaire. Les deux branches convergent vers la journalisation et la notification du demandeur.

Conclusion

Ce chapitre a défini l'architecture du système, son modèle de classes et les scénarios dynamiques de ses fonctionnalités clés. Le chapitre suivant présente la **réalisation** : l'environnement de travail et les interfaces de l'application.

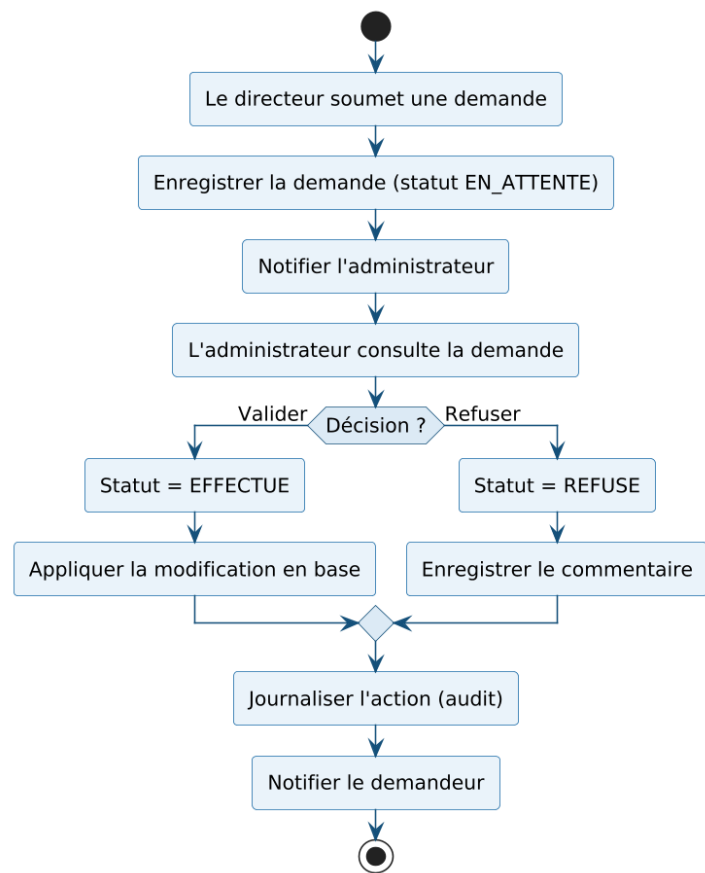


Figure 3.12 – Diagramme d'activité – Traitement d'une demande

Chapitre 4

Réalisation

Introduction

Ce chapitre présente la **réalisation** de l'application. Il décrit d'abord l'environnement de travail — matériel, logiciels et technologies mobilisés — puis illustre, au moyen de captures d'écran, les principales interfaces développées.

4.1 Environnement de travail

4.1.1 Environnement matériel

Le développement a été réalisé sur un poste de travail standard : ordinateur portable doté d'un processeur Intel Core i5, de 16 Go de mémoire vive et du système d'exploitation Windows 11.

4.1.2 Outil de rédaction du rapport

Le présent rapport a été rédigé avec **L^AT_EX**, système de composition de documents particulièrement adapté aux écrits scientifiques et techniques.



Figure 4.1 – Logo de « L^AT_EX »

4.1.3 Environnement logiciel

Plusieurs outils et technologies ont été mobilisés pour la réalisation du projet.

Java (JDK 21) : langage et plateforme d'exécution sur lesquels repose l'ensemble de l'application.



Figure 4.2 – Logo de « Java (JDK 21) »

Jakarta EE 10 : ensemble de spécifications d'entreprise (Servlets, JSP, EJB, JPA) structurant l'application en couches.



Figure 4.3 – Logo de « Jakarta EE »

Hibernate : implémentation de JPA assurant le mapping objet-relationnel entre les entités Java et la base Oracle.



Figure 4.4 – Logo de « Hibernate »

Oracle Database : système de gestion de base de données relationnelle hébergeant l'ensemble des données opérationnelles et les vues décisionnelles.



Figure 4.5 – Logo d'« Oracle Database »

WildFly / JBoss : serveur d'applications Jakarta EE sur lequel l'application est déployée sous forme d'archive WAR.



Figure 4.6 – Logo de « WildFly / JBoss »

Apache Maven : outil de gestion des dépendances et d'automatisation du *build* (production du WAR).



Figure 4.7 – Logo d'« Apache Maven »

IntelliJ IDEA : environnement de développement intégré utilisé pour le développement, le débogage et le déploiement de l'application.



Figure 4.8 – Logo d'« IntelliJ IDEA »

Git et GitHub : système de gestion de versions et plateforme d'hébergement du code source.



Figure 4.9 – Logos de « Git » et « GitHub »

ApexCharts : bibliothèque JavaScript de visualisation utilisée pour les graphiques du tableau de bord embarqué.



Figure 4.10 – Logo d'« ApexCharts »

Microsoft Power BI : outil de *reporting* décisionnel connecté aux vues analytiques de la base Oracle.



Figure 4.11 – Logo de « Microsoft Power BI »

Python : langage utilisé pour la chaîne **ETL** et la **segmentation** des agences, au moyen des bibliothèques **pandas** (manipulation des données) et **scikit-learn** (apprentissage automatique).



Figure 4.12 – Logos de « Python », « pandas » et « scikit-learn »

4.2 Interfaces de l'application

Cette section illustre les principales interfaces réalisées, présentées dans l'ordre des modules fonctionnels.

4.2.1 Authentification

La figure 4.13 présente la page d'authentification : l'utilisateur y saisit son identifiant et son mot de passe pour accéder à l'application.

4.2.2 Gestion des agences et des employés

Les figures 4.14 et 4.15 illustrent les interfaces de gestion des agences et des employés.

4.2.3 Gestion des clients et des objectifs

Les figures 4.16 et 4.17 illustrent les interfaces de gestion des clients et de suivi des objectifs commerciaux.

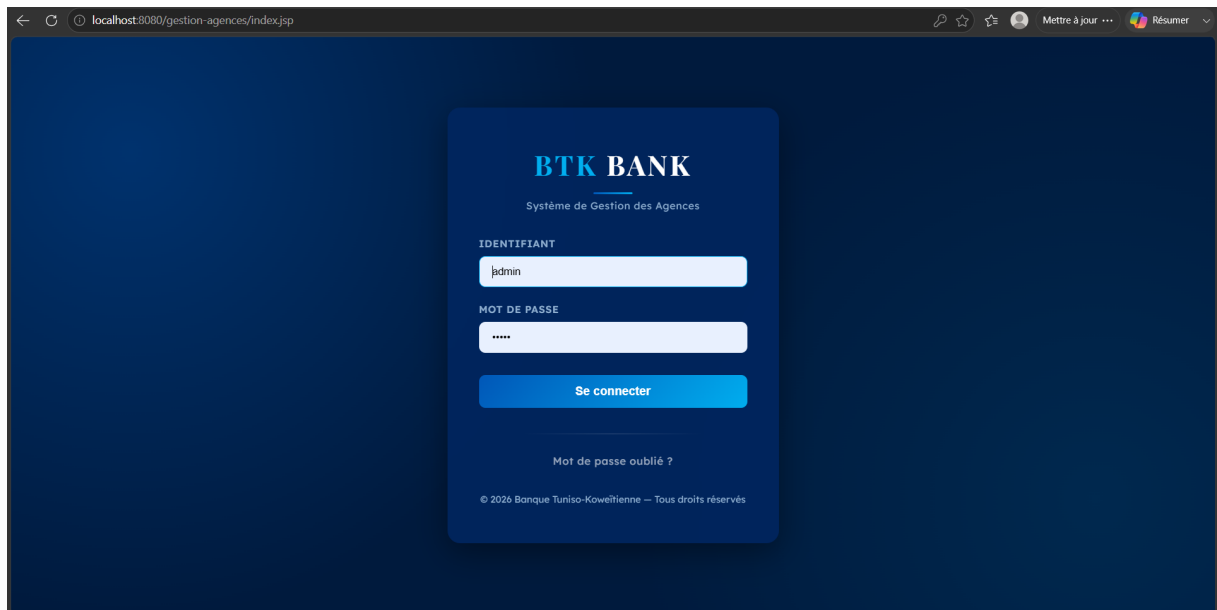


Figure 4.13 – Interface d'authentification

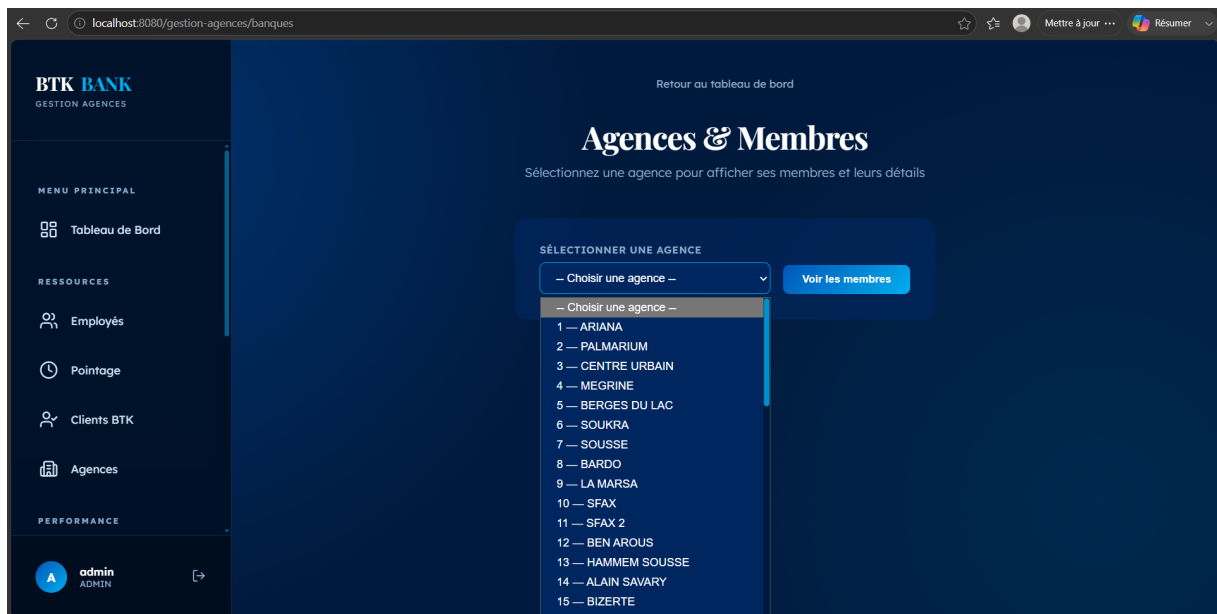


Figure 4.14 – Interface de gestion des agences (sélection et consultation des membres)

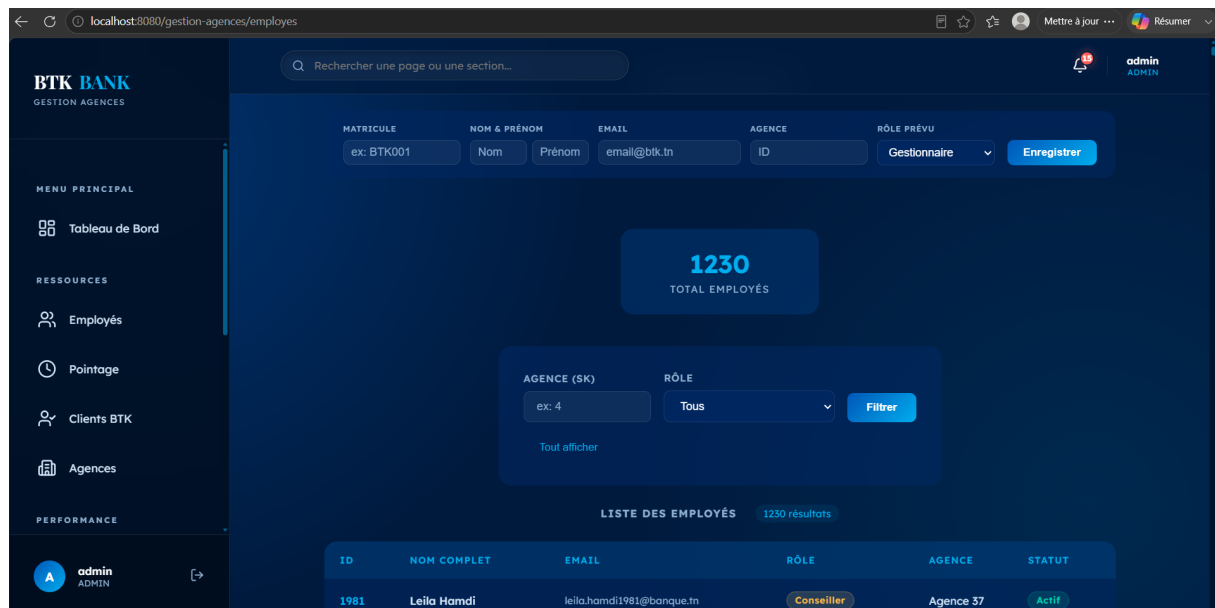


Figure 4.15 – Interface de gestion des employés (ajout, filtres et liste)

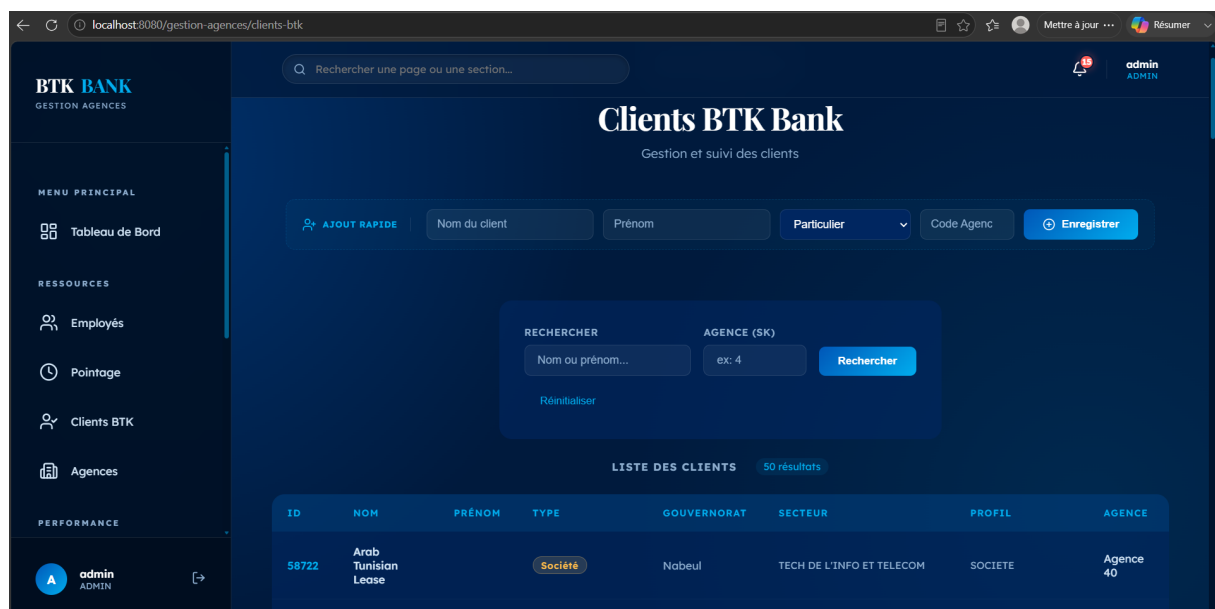


Figure 4.16 – Interface de gestion des clients (portefeuille BTK)

Objectifs Commerciaux
Suivi des objectifs par agence et gestionnaire — Total : 376

AGENCE (SK) : ex: 4
GESTIONNAIRE (SK) : ex: 76
Filtrer
Tout afficher

AGENCE	GESTIONNAIRE	SITUATION	EER PART.	EER H. PART.	CPT CHÈQUES	CPT ÉPARGNES	CRÉDITS CONSO	CRÉDITS IMMO	DAV PP
Agence 1	Gest. 67	Detaché	36.0	24.0	27.0	31.0	750000.0	550000.0	144000.0
Agence 1	Gest. 152	CIVP	60.0	24.0	45.0	51.0	900000.0	550000.0	120000.0
Agence 1	Gest. 223	Detaché	96.0	12.0	72.0	82.0	900000.0	550000.0	96000.0
Agence 1	Gest. 307	Permanent	48.0	0.0	36.0	41.0	288000.0	112000.0	48000.0
Agence 1	Gest. 0	Permanent	36.0	0.0	27.0	31.0	0.0	0.0	0.0

Figure 4.17 – Suivi des objectifs commerciaux par agence et gestionnaire

4.2.4 Pointage des employés

La figure 4.18 présente l'interface du module de pointage : les cartes de synthèse (présents, retards, absents, taux de présence), le pointage du jour (boutons « Pointer l'arrivée / le départ ») et le formulaire de saisie ou de correction manuelle.

Pointage des Employés
Présence quotidienne : arrivées, départs et statuts

mercredi 24 juin 2026

0 PRÉSENTS
0 RETARDS
0 ABSENTS
% TAUX DE PRÉSENCE

Mon pointage du jour
Vous n'avez pas encore pointé aujourd'hui.
Pointer l'arrivée
Pointer le départ

Saisie / Correction manuelle
EMPLOYÉ : Leila Hamdi
DATE : jj/mm/aaaa
STATUT : Présent
HEURE D'ARRIVÉE : --:--
HEURE DE DÉPART : --:--
COMMENTAIRE : Optionnel
Enregistrer le pointage

Figure 4.18 – Interface du module de pointage des employés

4.3 Déploiement de l'application

L'application est packagée par **Maven** sous forme d'une archive web `gestion-agences.war`, puis déployée sur le serveur d'applications **WildFly**. La persistance repose sur une **unité de persistance JPA** de type JTA, qui accède à la base Oracle via la source de données `java:/OracleDS` déclarée sur le serveur. Le fichier `persistence.xml` (extrait 4.1) déclare les entités du domaine et la connexion à la base `FREEPDB1`.

Listing 4.1 – Configuration de la persistance (`persistence.xml`)

```
1 <persistence-unit name="default" transaction-type="JTA">
2   <jta-data-source>java:/OracleDS</jta-data-source>
3   <class>entity.Utilisateur</class>
4   <class>entity.Agence</class>
5   <class>entity.BClient</class>
6   <class>entity.Objectif</class>
7   <class>entity.Pointage</class>
8   <class>entity.Gestionnaire</class>
9   <!-- ... autres entites ... -->
10  <properties>
11    <property name="jakarta.persistence.jdbc.driver" value="oracle.jdbc.
12      OracleDriver"/>
13    <property name="jakarta.persistence.jdbc.url"
14      value="jdbc:oracle:thin:@localhost:1521/FREEPDB1"/>
15    <property name="hibernate.dialect" value="org.hibernate.dialect.OracleDialect"/
16    >
17    <property name="hibernate.hbm2ddl.auto" value="none"/>
18  </properties>
19 </persistence-unit>
```

Le paramètre `hibernate.hbm2ddl.auto = none` indique que le schéma de la base est géré par les scripts SQL d'initialisation (annexe) et non régénéré par Hibernate, garantissant la maîtrise de la structure de la base en production.

4.4 Tests et validation

Afin de garantir la conformité de l'application aux besoins spécifiés, des **tests fonctionnels** ont été menés au fil du développement, module par module. Chaque fonctionnalité livrée a été éprouvée à travers des scénarios nominaux et des cas limites (identifiants invalides, accès non autorisé, pointage tardif), puis validée avec l'encadrement. Le tableau 4.1 récapitule les principaux scénarios de test et leur résultat.

Table 4.1 – Principaux scénarios de test fonctionnel

Module	Scénario testé	Résultat
Authentification	Connexion avec des identifiants valides puis invalides.	Conforme
Rôles	Accès à une fonction réservée à l'ADMIN par un USER.	Conforme
Agences	Création, modification et suppression d'une agence.	Conforme
Employés	Ajout d'un employé et rattachement à une agence.	Conforme
Clients	Affectation d'un client à un gestionnaire.	Conforme
Objectifs	Consultation de la production par agence et période.	Conforme
Pointage	Pointage au-delà de 9 h (détermination du statut).	Conforme (RETARD)
Demandes	Validation puis refus d'une demande avec journalisation.	Conforme
ETL	Exécution du pipeline et production du datamart.	Conforme
Décisionnel	Filtrage par district dans le rapport Power BI.	Conforme

L'ensemble des scénarios a produit le comportement attendu, ce qui confirme la **conformité fonctionnelle** de la solution aux besoins exprimés au chapitre 2.

Conclusion

Ce chapitre a présenté l'environnement de travail, les interfaces réalisées et la validation fonctionnelle des modules transactionnels de l'application. Le chapitre suivant est consacré au **volet décisionnel** : la chaîne ETL, le tableau de bord, les rapports Power BI et la segmentation des agences.

Chapitre 5

Volet décisionnel

Introduction

Ce chapitre présente le **volet décisionnel** de la solution. Il met en place une chaîne **ETL** qui consolide les données opérationnelles dans un **datamart**, restitue les indicateurs au moyen d'un tableau de bord embarqué et de rapports Power BI, puis introduit une brique d'**intelligence artificielle** : la segmentation des agences par apprentissage non supervisé.

5.1 Chaîne décisionnelle et processus ETL

Les données nécessaires au pilotage sont dispersées dans les tables opérationnelles et exprimées à des grains hétérogènes. Le processus **ETL** (*Extract – Transform – Load*) les consolide en un datamart agrégé par agence. La figure 5.1 présente cette chaîne décisionnelle.

Trois étapes s'enchaînent : **Extract** (lecture des sources AGENCE, B_UTILISATEURS, CLIENT_BTK, B_OBJECTIF, POINTAGE), **Transform** (nettoyage, typage et agrégation par agence) et **Load** (chargement du datamart en étoile). Ce datamart constitue une **source unique consolidée**, partagée par la restitution (tableau de bord et Power BI) et par la segmentation des agences présentée à la section 5.6.

5.2 Modèle en étoile de l'entrepôt

L'entrepôt est organisé selon une logique en **étoile** (figure 5.2) : autour de tables de faits (production, effectifs, clients, présence), des dimensions descriptives (agence, gestionnaire,

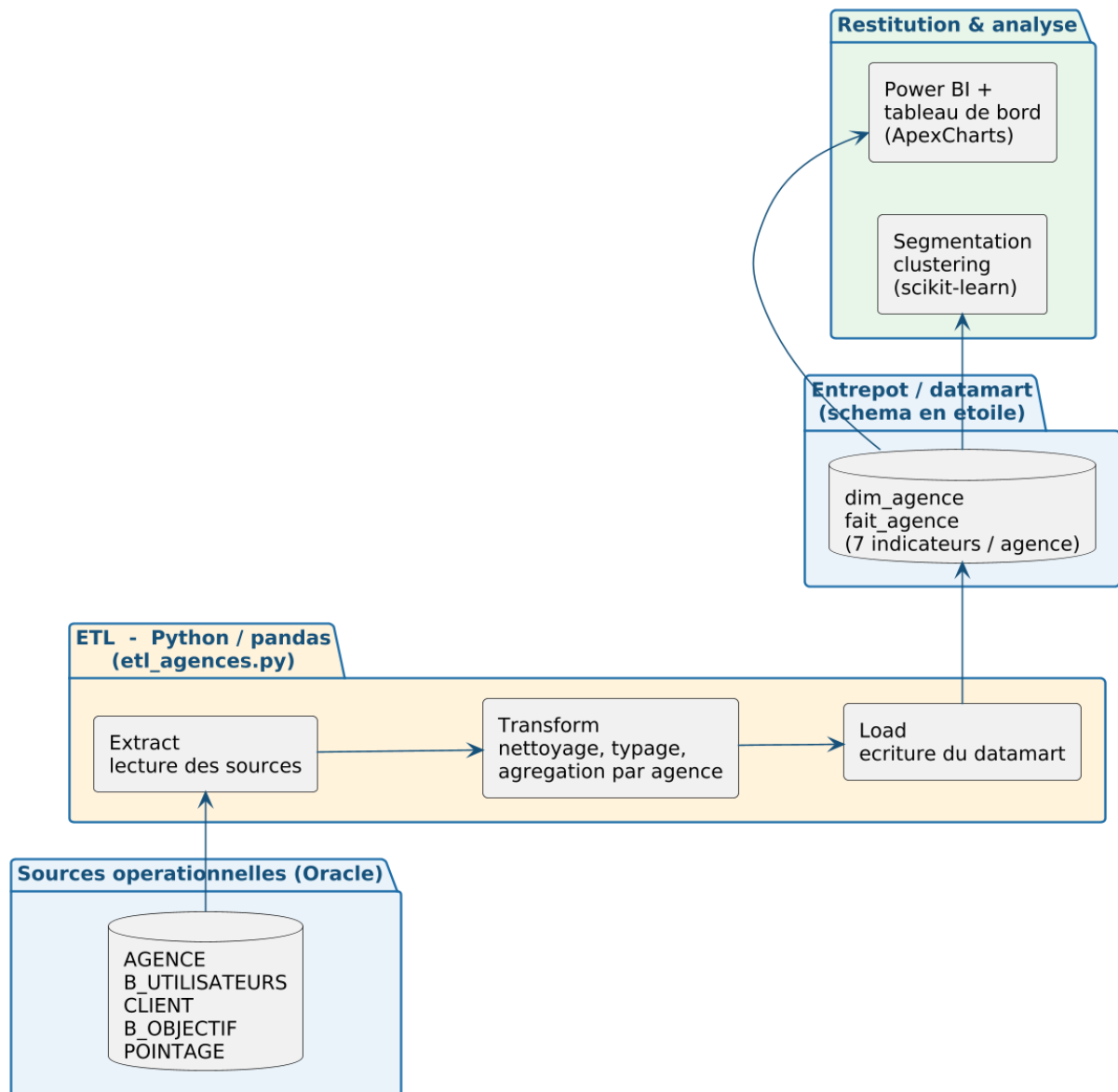


Figure 5.1 – Chaîne décisionnelle : du système opérationnel au datamart et à sa restitution

période, type de client) permettent l'analyse multidimensionnelle. Les vues V_BI_* matérialisent cette couche sémantique pour l'outil de restitution.

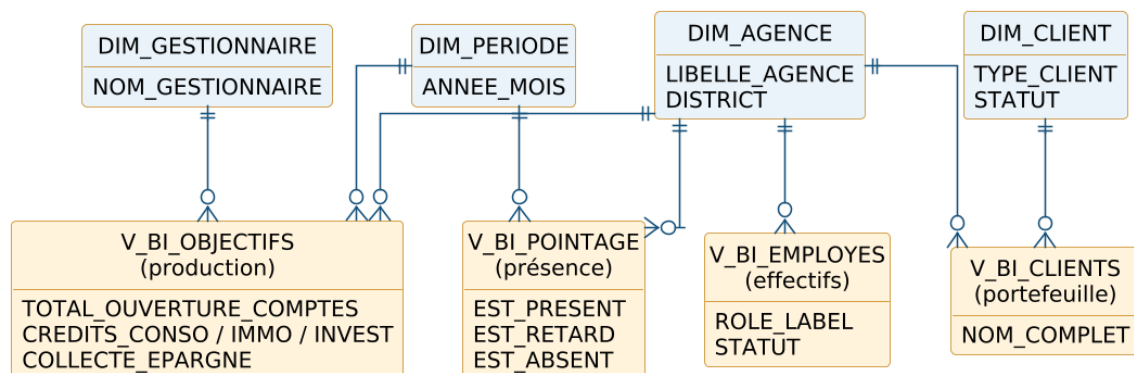


Figure 5.2 – Modèle décisionnel en étoile (vues V_BI_*)

Le tableau 5.1 récapitule les principaux indicateurs (KPI) retenus.

Table 5.1 – Indicateurs de performance (KPI) du volet décisionnel

Indicateur	Description	Source
Effectifs par agence	Nombre d'employés par agence	V_BI_EMPLOYES
Répartition des rôles	Gestionnaires / conseillers / hors commercial	V_BI_EMPLOYES
Portefeuille clients	Clients par type et par agence	V_BI_CLIENTS
Ouvertures de comptes	Comptes chèques + épargne + courants	V_BI_OBJECTIFS
Production de crédits	Conso, immobilier, investissement	V_BI_OBJECTIFS
Collecte d'épargne	Épargne additionnelle collectée	V_BI_OBJECTIFS
Taux de présence	Présents, retards et absences par agence et période	V_BI_POINTAGE

5.3 Mise en œuvre de la chaîne ETL en Python

5.3.1 Export des données sources

La chaîne ETL peut se connecter directement à Oracle ou, à défaut, consommer des fichiers CSV. Pour ce second mode, un script **SQLcl** (etl/export_sources.sql) exporte les cinq tables sources vers des CSV horodatés (extrait 5.1). Chaque SP00L redirige le résultat d'une requête vers un fichier, placé ensuite dans etl/source/.

Listing 5.1 – Export des sources vers CSV (SQLcl)


```

1 SET SQLFORMAT CSV
2 SPOOL agences.csv
3 SELECT SK_AGENCE, LIBELLE_AGENCE, DISTRICT FROM AGENCE;
4 SPOOL OFF
5
6 SPOOL objectifs.csv
7 SELECT SK_AGENCE,
8        NVL(SOUSCRIPTION_COMPTE_CHEQUES_OA,0)
9          + NVL(SOUSCRIPTION_COMPTE_EPARGNES_OA,0)
10         + NVL(SOUSCRIPTION_COMPTE_COURANTS_OA,0) AS COMPTES,
11        NVL(PRODUCTION_CREDITS_CONSO_OA,0)
12          + NVL(PRODUCTION_CREDITS_IMMO_OA,0)
13         + NVL(PRODUCTION_CREDITS_INVESTISSEMENT_OA,0) AS CREDITS,
14        NVL(EPARGNE_ADD_OA,0) AS EPARGNE
15 FROM B_OBJECTIF;
16 SPOOL OFF

```

5.3.2 Agrégation et chargement

La chaîne ETL est implémentée en **Python** (script `etl_agences.py`) à l'aide de la bibliothèque **pandas** [1]. Elle est organisée en trois fonctions : `extract()` lit les données sources (connexion Oracle via le connecteur `oracledb` ou, à défaut, export CSV); `transform()` nettoie et **agrège les données par agence** pour produire les sept indicateurs de pilotage; `load()` écrit le datamart en étoile (`dim_agence`, `fait_agence`) et le fichier réutilisé par la segmentation. L'extrait 5.2 illustre le cœur de l'agrégation.

Listing 5.2 – Agrégation des indicateurs par agence (étape Transform)

```

1 # effectif et nombre de gestionnaires par agence
2 eff = employees.groupby("SK_AGENCE").agg(
3     effectif=("SK_UTILISATEUR", "count"),
4     nb_gestionnaires=("EST_GESTIONNAIRE", "sum")).reset_index()
5 # portefeuille clients
6 cli = clients.groupby("SK_AGENCE").size().reset_index(name="nb_clients")
7 # production (comptes, credits, epargne)
8 obj = objectifs.groupby("SK_AGENCE").agg(
9     total_comptes=("COMPTES", "sum"),

```

```

10 production_credits=("CREDITS", "sum"),
11 collecte_epargne=("EPARGNE", "sum")).reset_index()
12 # taux de presence issu du pointage
13 poi = pointages.assign(
14     present=pointages["STATUT"].isin(["PRESENT", "RETARD"]).astype(int)) \
15     .groupby("SK_AGENCE").agg(taux_presence=("present", "mean")).reset_index()

```

L'exécution du script produit le datamart sous `etl/entrepot/` (`dim_agence.csv`, `fait_agence.csv`) ainsi que le fichier consolidé `clustering/data/agences.csv`. Le pipeline est **rejouable** et journalise le nombre d'enregistrements traités à chaque étape.

5.4 Tableau de bord et restitution Power BI

Le tableau de bord embarqué (`home.jsp`, alimenté par le `HomeServlet`) comporte une rangée de **cartes d'indicateurs** (employés, clients, présents du jour, taux de présence, demandes en attente) puis des **graphiques** ApexCharts (effectif par agence, répartition des rôles, présence du jour). En complément, les vues `V_BI_*` sont connectées à **Power BI** pour des rapports interactifs. La figure 5.3 présente le tableau de bord embarqué.

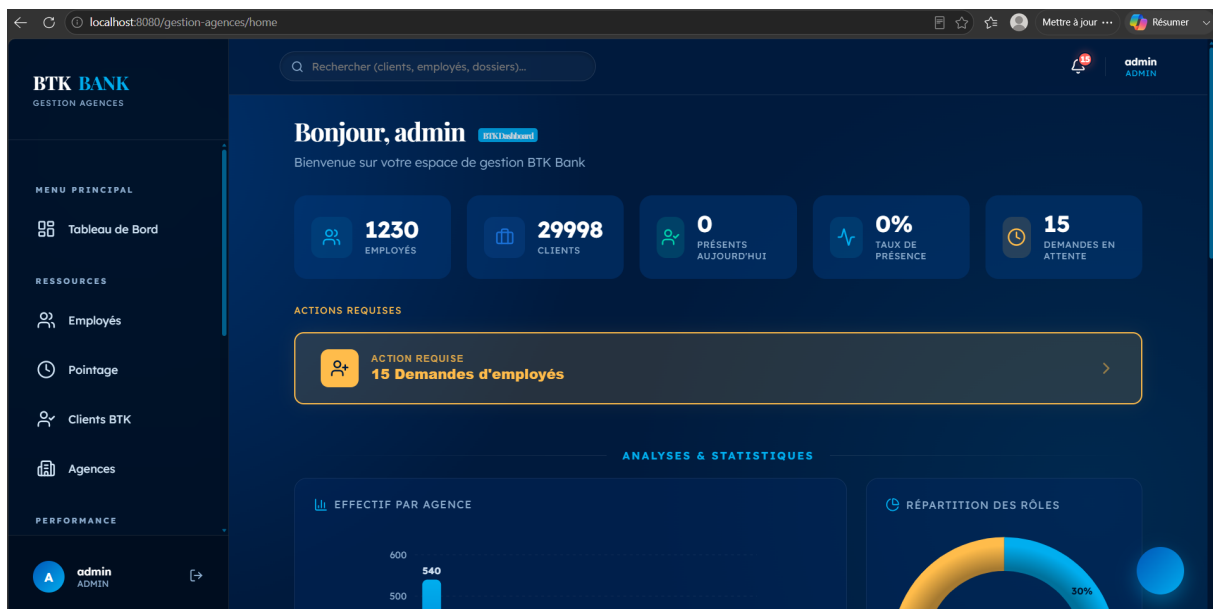


Figure 5.3 – Tableau de bord décisionnel embarqué de l'application

5.5 Rapport décisionnel sous Power BI

Au-delà du tableau de bord embarqué, un **rapport décisionnel dédié** a été développé sous **Microsoft Power BI**, connecté aux données consolidées. Il est organisé en **quatre pages** analytiques thématiques — *Résumé*, *Clients*, *Commercial* et *Présence* — reliées par une barre de navigation, chacune munie de **segments** de filtrage (district, période, type de client) qui recalculent l'ensemble des visuels d'un simple clic. La **page de synthèse** (*Résumé*) regroupe les indicateurs clés du réseau (clients, production de crédits, taux de présence, ouvertures de comptes), la répartition des rôles et des présences, l'évolution mensuelle de la production et un classement des agences. Les trois pages suivantes approfondissent respectivement l'analyse des clients, la performance commerciale et l'assiduité.

5.5.1 Indicateurs et mesures (DAX)

Les indicateurs restitués sont calculés au moyen de **mesures DAX** définies sur le modèle de données. Le tableau 5.2 en récapitule les principales.

Table 5.2 – Principales mesures DAX du rapport Power BI

Mesure	Définition
Nb Employés	Nombre distinct d'employés (SK_UTILISATEUR).
Nb Clients	Nombre distinct de clients (SK_CLIENT).
Total Comptes	Somme des ouvertures de comptes (chèques + épargne + courants).
Total Crédits	Somme de la production de crédits (conso + immobilier + investissement).
Total Épargne	Épargne additionnelle collectée.
Total Packs & Cartes	Souscriptions de packs (particulier, pro) et de cartes.
Taux Présence	Part des pointages au statut « présent » ou « retard ».
Nb Présences / Retards / Absences	Nombre de pointages par statut.
Rôle (colonne calculée)	Gestionnaire / Conseiller / Hors commercial, selon EST_GESTIONNAIRE et l'agence.

5.5.2 Analyse des clients

La page *Clients* (figure 5.4) segmente le portefeuille selon plusieurs axes : le **type de client**, le **sexe**, la **région** et le **gouvernorat** (histogramme des principaux gouvernorats), complétés par un classement des agences selon leur nombre de clients. Elle offre une lecture **géographique et démographique** du portefeuille, absente du tableau de bord opérationnel.

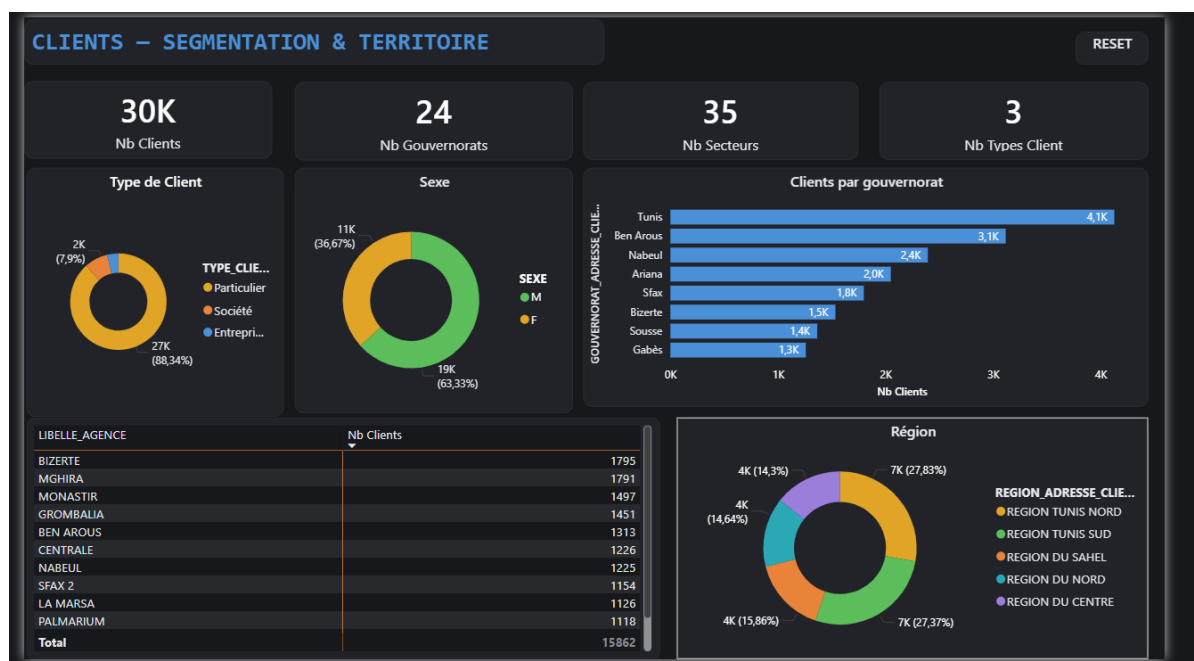


Figure 5.4 – Rapport Power BI — page « Clients » (segmentation et territoire)

5.5.3 Performance commerciale

La page *Commercial* (figure 5.5) détaille les **objectifs** : les ouvertures de comptes par type, la production de crédits par nature, l'effort d'équipement (EER), l'**évolution mensuelle** de la production et un classement des agences par production. Elle constitue l'outil de **pilotage commercial** du réseau.

5.5.4 Suivi de la présence

La page *Présence* (figure 5.6) suit l'**assiduité** des employés : le taux de présence, la répartition des statuts (présent, retard, absent) et de la source de pointage (automatique, manuel), l'**évolution mensuelle** du taux de présence et la liste des **agences à surveiller** (présence la plus faible). Elle prolonge le module de pointage par une vue **ressources humaines** consolidée.

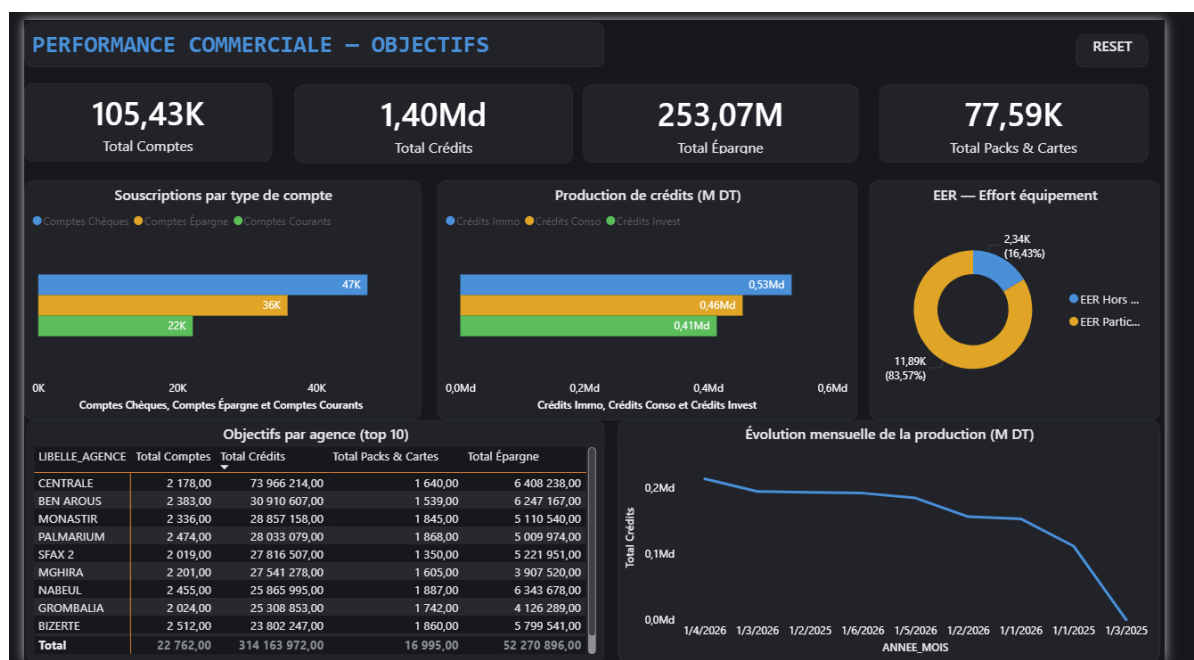


Figure 5.5 – Rapport Power BI — page « Commercial » (objectifs et production)

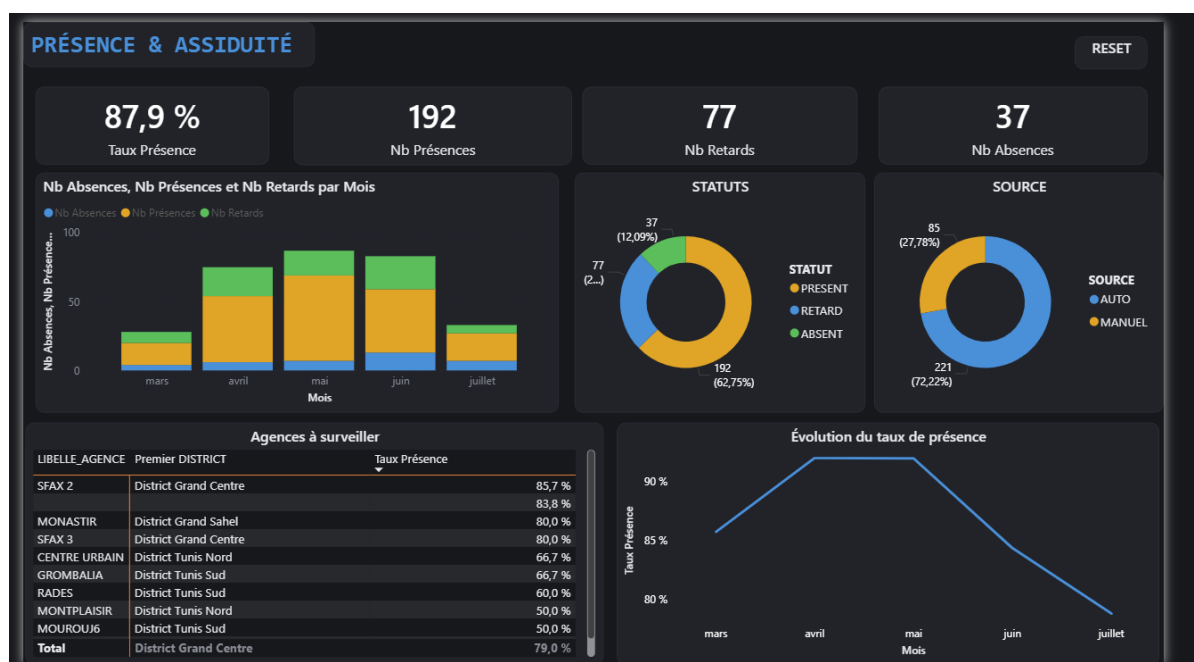


Figure 5.6 – Rapport Power BI — page « Présence » (assiduité et suivi RH)

5.5.5 Navigation et interactivité

Les quatre pages sont reliées par un **navigateur** qui joue le rôle de menu. Chaque page est équipée de **segments** (district, période, type de client) et d'un bouton **RÉINITIALISER** : un clic sur une valeur recalcule instantanément l'ensemble des visuels de la page (*filtrage croisé*), et la sélection d'une part d'un graphique filtre à son tour les autres. Cette **interactivité** transforme le rapport en un véritable outil d'exploration : le décideur peut, par exemple, isoler un district puis observer simultanément le portefeuille, la production et l'assiduité correspondants. Un **thème sombre** homogène assure la lisibilité et la cohérence visuelle de l'ensemble.

5.6 Segmentation des agences par apprentissage non supervisé

Cette dernière section introduit une brique d'**intelligence artificielle** : la segmentation des agences par **apprentissage non supervisé** (*clustering*). À partir du datamart produit par la chaîne ETL, elle regroupe automatiquement les agences en profils homogènes afin d'appuyer le ciblage, l'allocation des ressources et la définition d'objectifs différenciés.

5.6.1 Données et variables

Les données ne sont pas extraites directement de la base opérationnelle : elles proviennent du **datamart** produit par la chaîne ETL (table de faits `fait_agence`). Chaque agence y est décrite par sept indicateurs agrégés (tableau 5.3), ce qui garantit la cohérence entre le tableau de bord, Power BI et l'analyse non supervisée.

Remarque. En l'absence d'un export des données de production, la segmentation présentée ici est **illustrée sur un jeu de données représentatif** du réseau, généré par la chaîne ETL pour reproduire la structure et les ordres de grandeur des agences ; le pipeline bascule automatiquement sur les données réelles dès qu'un export est fourni (`etl/source/`). Les résultats ci-dessous illustrent donc la *démarche* et restent reproductibles sur les données réelles.

Table 5.3 – Variables de segmentation (par agence)

Variable	Description
effectif	Nombre d'employés de l'agence
nb_gestionnaires	Nombre de gestionnaires
nb_clients	Taille du portefeuille clients
total_comptes	Total des ouvertures de comptes
production_credits	Production de crédits
collecte_epargne	Collecte d'épargne
taux_presence	Taux de présence (issu du pointage)

5.6.2 Prétraitement

Les variables étant exprimées dans des ordres de grandeur très différents (un effectif de quelques dizaines face à des milliers de comptes), une **standardisation** (centrage-réduction, `StandardScaler`) est appliquée. Sans elle, les variables de forte amplitude domineraient le calcul des distances et fausseraient le regroupement.

5.6.3 Détermination du nombre de clusters

Le nombre de clusters k étant inconnu a priori, il est déterminé en combinant la **méthode du coude** (inertie intra-cluster) et le **score de silhouette** (figure 5.7).

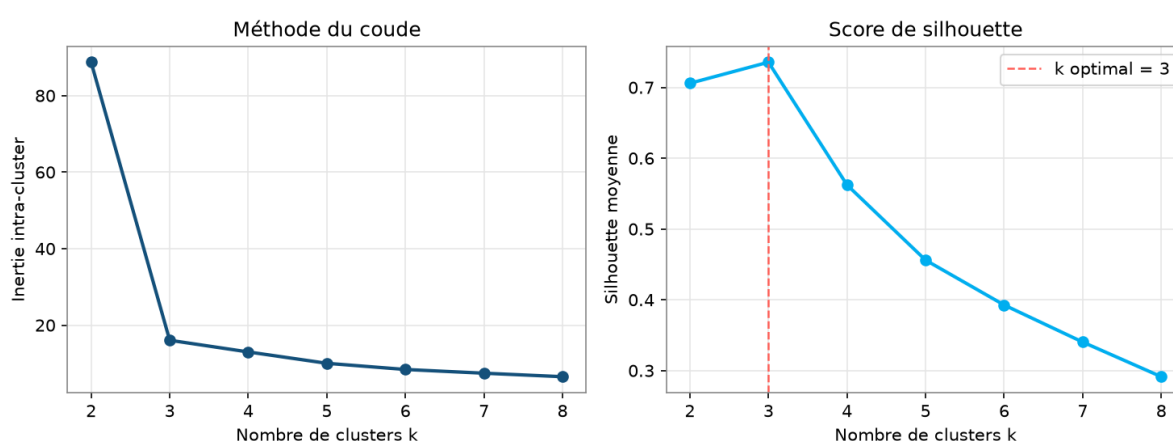


Figure 5.7 – Choix du nombre de clusters : méthode du coude et score de silhouette

Les deux indicateurs concordent : l'inertie présente un **coude marqué** à $k = 3$ et le score de silhouette y atteint son **maximum** ($\approx 0,74$). La segmentation en **trois clusters** est donc retenue.

5.6.4 Comparaison des modèles de clustering

Quatre algorithmes ont été comparés : **K-Means**, **classification ascendante hiérarchique** (agglomératif), **modèle de mélange gaussien** (GMM) et **DBSCAN**, à l'aide de trois indices internes (tableau 5.4, figure 5.8).

Table 5.4 – Comparaison des modèles de clustering

Modèle	Nb clusters	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑
K-Means	3	0,736	0,350	391
Agglomératif	3	0,736	0,350	391
GMM	3	0,736	0,350	391
DBSCAN	3	0,736	0,350	391

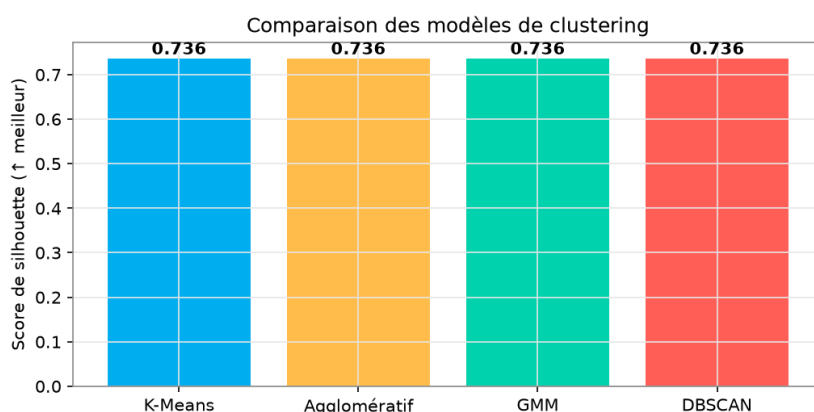


Figure 5.8 – Comparaison des modèles selon le score de silhouette

Les quatre algorithmes **convergent vers la même segmentation** : ils identifient les trois mêmes clusters et obtiennent des indices identiques (silhouette 0,736, Davies-Bouldin 0,350, Calinski-Harabasz 391). Cette concordance, y compris pour des approches de nature différente (partitionnement, hiérarchique, probabiliste et à base de densité), confirme la **robustesse** des trois segments. Le **K-Means** est retenu : il atteint le meilleur score tout en restant simple et interprétable.

5.6.5 Résultats et interprétation

La figure 5.9 projette les agences dans le plan des deux premières composantes principales (PCA), colorées selon leur cluster ; la première composante, qui résume la *taille* de l'agence,

explique l'essentiel de la variance. Les trois segments sont nettement séparés.

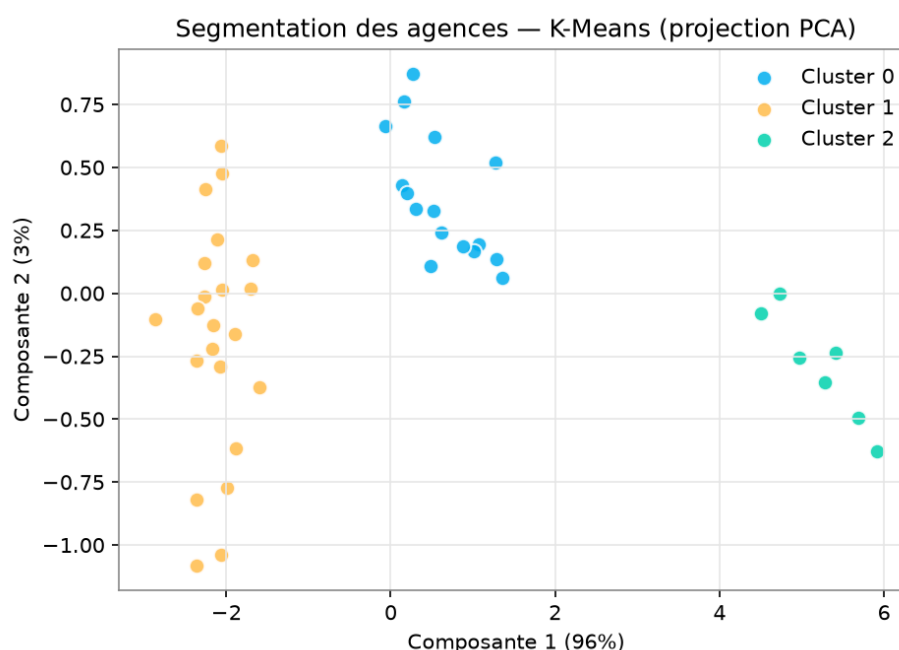


Figure 5.9 – Segmentation des agences (K-Means) projetée par PCA

Le profil moyen de chaque segment (tableau 5.5) permet de les caractériser.

Table 5.5 – Profil moyen des segments d'agences

Segment	Nb	Effectif	Clients	Comptes	Prod. crédits	Présence
Grandes agences	7	116	908	2 333	1734	0,98
Moyennes agences	16	54	438	1 076	823	0,94
Petites agences	22	21	160	431	321	0,87

On distingue ainsi un segment de **grandes agences** (pôles majeurs à fort effectif et forte production), un segment d'**agences moyennes** (activité intermédiaire) et un segment de **petites agences** (effectif et activité réduits, taux de présence plus faible).

5.6.6 Apport décisionnel

Cette segmentation constitue un outil d'**aide à la décision** : elle permet d'adapter les actions à chaque profil. Le tableau 5.6 décline des recommandations de pilotage par segment.

En résumé, la segmentation permet d'**allouer les ressources** selon la taille, de **différencier les objectifs** et d'**accompagner spécifiquement** les agences les plus fragiles. Elle prolonge le volet décisionnel et illustre l'apport de l'**intelligence artificielle** au pilotage du réseau.

Table 5.6 – Recommandations de pilotage par segment

Segment	Recommandations
Grandes agences	Capitaliser sur la performance ; fixer des objectifs ambitieux ; en faire des agences de référence et diffuser leurs bonnes pratiques.
Moyennes agences	Fort potentiel de croissance ; renforcer l'effort d'équipement (EER) et la collecte ; assigner des objectifs progressifs.
Petites agences	Accompagnement ciblé ; suivi rapproché de l'assiduité (taux de présence plus faible) ; mutualisation des ressources et montée en compétence.

Conclusion

Ce chapitre a doté la solution d'un volet décisionnel complet : une chaîne ETL alimente un datamart consolidé, restitué par le tableau de bord et par Power BI, et exploité par une segmentation des agences qui fait émerger trois profils cohérents et exploitables pour la décision.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et de réaliser, au sein de la **BTK – Banque Tuniso-Koweïtienne**, une application web de gestion des agences bancaires adossée à un volet décisionnel, conduite selon une démarche itérative et incrémentale. La solution centralise la gestion des agences, des employés, des clients et des objectifs, assure le **pointage** des employés, encadre les opérations sensibles par des circuits de validation et restitue des indicateurs d'aide à la décision via un tableau de bord et Power BI. Elle intègre en outre une chaîne **ETL** et une brique d'**intelligence artificielle** : la segmentation des agences par apprentissage non supervisé (clustering).

Ce travail a été l'occasion de mettre en pratique les compétences de la formation — modélisation UML, développement Jakarta EE, base de données Oracle, chaîne décisionnelle (ETL en Python et *reporting* Power BI) et initiation à l'**apprentissage automatique** — au croisement du développement logiciel et de la Business Intelligence.

Plusieurs **difficultés** ont jalonné le projet et ont constitué autant d'occasions d'apprentissage. La **consolidation** de données dispersées à des grains hétérogènes a nécessité une chaîne ETL robuste et rejouable. L'articulation entre le système opérationnel Oracle et l'outil de restitution Power BI a demandé une attention particulière à la **modélisation décisionnelle** (schéma en étoile, mesures) et à la préparation des données. Enfin, la mise au point de la segmentation a exigé une démarche méthodique de choix et de comparaison de modèles afin d'en garantir la robustesse. Ces défis ont renforcé une compétence clé : **transformer une donnée brute en information utile à la décision**.

Plusieurs perspectives d'évolution peuvent être envisagées :

- le chiffrement (hachage) des mots de passe et le renforcement de la sécurité ;
- l'**industrialisation** de la chaîne ETL : ordonnancement automatique et historisation incrémentale des indicateurs ;
- l'enrichissement par des analyses prédictives (prévision des objectifs) ;
- l'exposition d'une API REST pour l'interopérabilité avec d'autres systèmes ;
- le développement d'une application mobile de consultation des indicateurs.

Nétographie

- [1] The pandas development team, "pandas – python data analysis library," 2025. <https://pandas.pydata.org/docs/>.
- [2] Eclipse Foundation, "Jakarta ee 10 – documentation officielle," 2025. <https://jakarta.ee/>.
- [3] Red Hat, "Hibernate orm – documentation," 2025. <https://hibernate.org/orm/documentation/>.
- [4] Oracle Corporation, "Oracle database – documentation," 2025. <https://docs.oracle.com/en/database/>.
- [5] Red Hat, "Wildfly application server – documentation," 2025. <https://docs.wildfly.org/>.
- [6] Microsoft, "Microsoft power bi – documentation," 2025. <https://learn.microsoft.com/power-bi/>.
- [7] ApexCharts, "Apexcharts.js – documentation," 2025. <https://apexcharts.com/docs/>.
- [8] Apache Software Foundation, "Apache maven – documentation," 2025. <https://maven.apache.org/guides/>.
- [9] BTK Bank, "Banque tuniso-koweïtienne – site officiel," 2025. <https://www.btknet.com/>.
- [10] scikit-learn developers, "scikit-learn – machine learning in python," 2025. <https://scikit-learn.org/stable/>.

Annexe A

Annexes

Cette annexe rassemble des extraits du code réel du projet : la chaîne ETL, la segmentation par apprentissage automatique et une vue décisionnelle SQL.

A.1 Chaîne ETL — extraction, transformation, chargement (Python)

Extraits du script `etl/etl_agences.py`. La fonction `extract_oracle` lit les tables opérationnelles; `transform` agrège les indicateurs par agence; `load` écrit le datamart en étoile.

Listing A.1 – Extraction des sources depuis Oracle

```
1 def extract_oracle(dsn=None, user=None, password=None):
2     import oracledb
3     cn = oracledb.connect(
4         user=user or os.environ.get("BTK_DB_USER", "system"),
5         password=password or os.environ.get("BTK_DB_PWD", ""),
6         dsn=dsn or os.environ.get("BTK_DB_DSN", "localhost:1521/FREEPDB1"))
7     ag = pd.read_sql("SELECT SK_AGENCE, LIBELLE_AGENCE, DISTRICT FROM AGENCE", cn)
8     emp = pd.read_sql("SELECT SK_UTILISATEUR, SK_AGENCE, EST_GESTIONNAIRE "
9                       "FROM B_UTILISATEURS", cn)
10    cli = pd.read_sql("SELECT SK_CLIENT, SK_AGENCE FROM CLIENT_BTK", cn)
11    obj = pd.read_sql(
12        "SELECT SK_AGENCE, "
13        " NVL(SOUSCRIPTION_COMPTE_CHEQUES_OA,0)+NVL(SOUSCRIPTION_COMPTE_EPARGNES_OA,0)"
14        " +NVL(SOUSCRIPTION_COMPTE_COURANTS_OA,0) AS COMPTES, "
15        " NVL(PRODUCTION_CREDITS_CONSO_OA,0)+NVL(PRODUCTION_CREDITS_IMMO_OA,0)"
16        " +NVL(PRODUCTION_CREDITS_INVESTISSEMENT_OA,0) AS CREDITS, "
17        " NVL(EPARGNE_ADD_OA,0) AS EPARGNE FROM B_OBJECTIF", cn)
```

```

18 poi = pd.read_sql("SELECT P.SK_UTILISATEUR, U.SK_AGENCE, P.STATUT "
19                  "FROM POINTAGE P JOIN B_UTILISATEURS U "
20                  "ON P.SK_UTILISATEUR = U.SK_UTILISATEUR", cn)
21
22 cn.close()
23
24 return ag, emp, cli, obj, poi

```

Listing A.2 – Transformation (agrégation par agence) et chargement du datamart

```

1 def transform(agences, employes, clients, objectifs, pointages):
2     eff = employes.groupby("SK_AGENCE").agg(
3         effectif=("SK_UTILISATEUR", "count"),
4         nb_gestionnaires=("EST_GESTIONNAIRE", "sum")).reset_index()
5     cli = clients.groupby("SK_AGENCE").size().reset_index(name="nb_clients")
6     obj = objectifs.groupby("SK_AGENCE").agg(
7         total_comptes=("COMPTES", "sum"),
8         production_credits=("CREDITS", "sum"),
9         collecte_epargne=("EPARGNE", "sum")).reset_index()
10    poi = pointages.assign(
11        present=pointages["STATUT"].isin(["PRESENT", "RETARD"]).astype(int) \
12        .groupby("SK_AGENCE").agg(taux_presence=("present", "mean")).reset_index()
13    dm = agences.merge(eff, on="SK_AGENCE", how="left") \
14        .merge(cli, on="SK_AGENCE", how="left") \
15        .merge(obj, on="SK_AGENCE", how="left") \
16        .merge(poi, on="SK_AGENCE", how="left")
17    return dm.rename(columns={"LIBELLE_AGENCE": "agence"})
18
19 def load(dm):
20     dim_agence = dm[["SK_AGENCE", "agence", "DISTRICT"]]
21     fait_agence = dm[["SK_AGENCE"] + FEATURES]
22     dim_agence.to_csv(os.path.join(ENTREPOT, "dim_agence.csv"), index=False)
23     fait_agence.to_csv(os.path.join(ENTREPOT, "fait_agence.csv"), index=False)
24     dm[["agence"] + FEATURES].to_csv(DATAMART, index=False)

```

A.2 Segmentation des agences (Python, scikit-learn)

Extraits du script `clustering/segmentation_agences.py` : la standardisation des variables et la détermination du nombre optimal de clusters (méthode du coude combinée au score de silhouette).

Listing A.3 – Choix du nombre de clusters (coude + silhouette)

```
1 from sklearn.preprocessing import StandardScaler
2 from sklearn.cluster import KMeans
3 from sklearn.metrics import silhouette_score
4
5 X = StandardScaler().fit_transform(df[FEATURES]) # centrage-reduction
6
7 def choisir_k(X, kmax=8):
8     ks = list(range(2, kmax + 1))
9     inerties, silhouettes = [], []
10    for k in ks:
11        km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
12        inerties.append(km.inertia_) # methode du coude
13        silhouettes.append(silhouette_score(X, km.labels_))
14    return ks[int(np.argmax(silhouettes))] # k au meilleur silhouette
```

Listing A.4 – Comparaison des modèles et profilage des segments

```
1 modeles = {
2     "K-Means": KMeans(n_clusters=k, n_init=10, random_state=42),
3     "Agglomeratif": AgglomerativeClustering(n_clusters=k),
4     "GMM": GaussianMixture(n_components=k, random_state=42),
5     "DBSCAN": DBSCAN(eps=1.2, min_samples=3),
6 }
7 for nom, modele in modeles.items():
8     labels = modele.fit_predict(X)
9     sil = silhouette_score(X, labels)
10    db = davies_bouldin_score(X, labels)
11    ch = calinski_harabasz_score(X, labels)
12    print(nom, round(sil, 3), round(db, 3), round(ch))
13 # profil moyen de chaque segment
14 df["cluster"] = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X)
15 profils = df.groupby("cluster")[FEATURES].mean()
```

A.3 Vue décisionnelle (SQL)

Exemple de vue analytique exposée à Power BI (`sql/v_bi_employees.sql`) : elle joint les employés à leur agence et calcule le rôle métier.

Listing A.5 – Vue décisionnelle V_BI_EMPLOYES

```
1 CREATE OR REPLACE VIEW V_BI_EMPLOYES AS
2 SELECT
3     U.SK_UTILISATEUR AS ID_EMPLOYE,
4     U.LIBELLE_UTILISATEUR AS NOM_COMPLET,
5     U.EMAIL_UTILISATEUR AS EMAIL,
6     CASE
7         WHEN U.EST_GESTIONNAIRE = 1 THEN 'Gestionnaire'
8         WHEN U.EST_GESTIONNAIRE = 0 AND U.SK_AGENCE = 31 THEN 'Hors Commercial'
9         ELSE 'Conseiller'
10    END AS ROLE_LABEL,
11    CASE WHEN U.EST_SUSPENDU = 1 THEN 'Suspendu' ELSE 'Actif' END AS STATUT,
12    A.LIBELLE_AGENCE AS NOM_AGENCE,
13    A.DISTRICT AS DISTRICT_AGENCE
14 FROM B_UTILISATEURS U
15 LEFT JOIN AGENCE A ON U.SK_AGENCE = A.SK_AGENCE;
```

A.4 Dépôt du code source

Le code source complet du projet — application Jakarta EE (entités, servlets, services, pages JSP), scripts SQL d’initialisation, chaîne ETL et segmentation Python, thème et modèle Power BI — est versionné sur le dépôt Git associé à ce rapport.